

一、Embeddings和向量数据库08:43

1. Embedding 18:21

1) 嵌入的定义 18:22

>> 今天的学习目标



什么是Embedding

- 什么是Embedding
- N-Gram
- Word Embedding
- 余弦相似度计算
- Embedding模型的选择
- MTED榜单
- 向量维度对模型性能的影响
- 神奇的“俄罗斯套娃”

向量数据库

- 什么是向量数据库
- FAISS, Elasticsearch, Milvus, Pinecone的特点
- 向量数据库与传统数据库的区别
- 如何将数据导入向量数据库
- Embedding与原数据导入Faiss
- 不同向量数据库的功能与性能比较

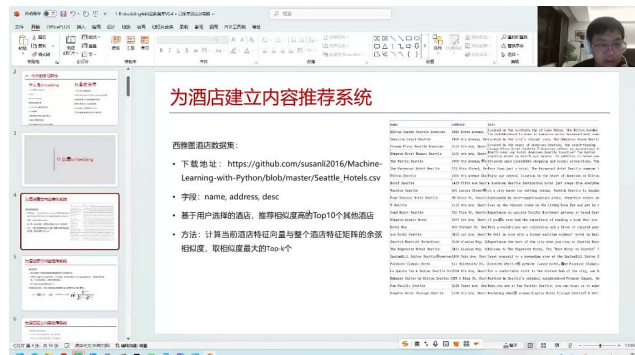
- 如何选择适合的Embedding模型
- 本质：将高维数据压缩到固定低维空间的过程
- 作用：通过体积压缩实现高效存储和计算
- 应用场景：信息存储管理、向量相似度计算（如余弦相似度）

2) Word Embedding 19:00

- 核心功能：将单词转化为向量表示
- 相似度计算：使用余弦相似度衡量向量间关系
- 模型选择维度：
 - 向量尺寸（1024/2048等）
 - 模型大小
 - 性能影响（不同维度对计算效率的影响）

3) 传统语义特征提取方法 20:33

- 学习目标与数据集介绍 20:36

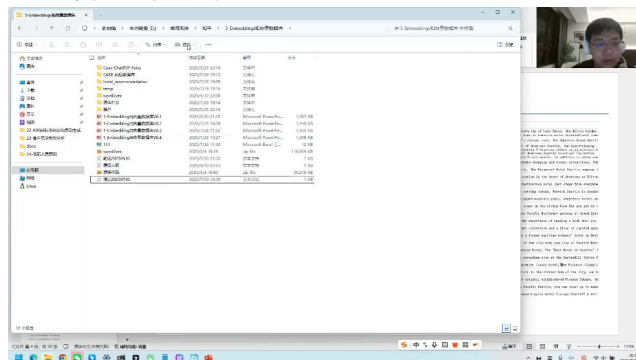


- 数据集：西雅图152家酒店数据（名称/地址/描述）
- 应用目标：基于文本相似度推荐Top10相似酒店
- 计算方法：特征向量与全量矩阵的余弦相似度计算
- 传统语义特征提取方法：分词 22:47
 - 基本单元：以单词为最小处理单位
 - 中文工具：结巴分词器<https://github.com/fxsjy/jieba>
 - 示例：
 - 句子A: "这个程序代码太乱那个代码规范" → ["这个", "程序", "代码", "太乱", "那个", "代码", "规范"]
 - 句子B: "这个程序代码不规范那个更规范" → ["这个", "程序", "代码", "不", "规范", "那个", "更", "规范"]

- 传统语义特征提取方法：特征值与向量 23:53
 - 特征构建：
 - 统计所有句子中的唯一单词作为特征维度
 - 计算每个句子在各维度上的出现频次
 - 向量表示：
 - 句子A向量：[1,1,2,1,1,1,0,0] (8维)
 - 句子B向量：[1,1,1,0,1,2,1,1] (8维)
- 传统语义特征提取方法：余弦相似度 25:46
 - 数学原理：
 - 计算公式： $\cos\theta = \frac{A \cdot B}{||A|| \times ||B||}$
 - 取值范围：[-1,1] (1表示完全一致，-1表示完全相反)
 - 实际计算：
 - 示例句子计算结果：0.738
 - 解释：接近1表示语义高度相似
- 传统语义特征提取方法：存在的问题 30:35
 - 关键缺陷：忽略词语顺序对语义的影响
 - 反例证明：
 - 调整语序后："这个程序代码规范那个更不规范"
 - 向量不变但实际语义相反
 - 根本原因：仅统计单词出现频次（一元语法）
- 传统语义特征提取方法：N元语法 32:40
 - 改进方案：引入词语组合特征
 - 具体方法：
 - 二元语法：统计相邻两个单词组合（如AB、BC）
 - 三元语法：统计相邻三个单词组合（如ABC、BCD）
 - 优势：能捕捉"不规范"与"规范不"等关键词序差异
 - 示例扩展：
 - 序列"a b c d e"
 - 二元特征：ab, bc, cd, de
 - 三元特征：abc, bcd, cde

4) 应用案例 34:17

- 例题：西雅图酒店数据集



- 数据准备：使用西雅图酒店数据集，包含152家酒店的名称、地址和描述信息
- 数据特征：主要分析酒店描述文本中的n-gram特征（一元、二元、三元语法）
- 文本特征提取

为酒店建立内容推荐系统



3 基于相似度和距离和相似度的酒店name, 推荐TOP10酒店

```
def recommend(name, cosine_similarities = cosine_similarities):
    recommended_hotels = []

    # 找到和酒店匹配的索引id
    idx = indices[indices == name].index[0]
    print(idx, 'idx')

    # 对id内的相似度按照从大到小进行排序
    score_series = pd.Series(cosine_similarities[idx]).sort_values(ascending=False)

    # 取相似度最大的前10个 (除了自己本身)
    top_10_indices = list(score_series.index[1:11])

    # 放到推荐列表+
    for i in top_10_indices:
        recommended_hotels.append(indices[idx][i])

    return recommended_hotels

print(recommend('Hilton Seattle Airport & Conference Center'))
print(recommend('The Beja Inn Seattle (Bed and Breakfast)'))
```

Side 4:

"Emassy Suites by Hilton Seattle Tacoma International Airport"
"Doubletree by Hilton Hotel Seattle Airport", "Seattle Airport
Hotel", "Hilton Seattle Sea-Tac Airport South", "Econo Lodge
SeaTac Airport North", "Four Points by Sheraton Downtown Seattle
Center", "Knights Inn Tukwila", "Econo Lodge Renton-Bellevue",
"Hampton Inn Seattle/Southwest", "Radisson Hotel Seattle
Airport"
Side 116
"11th Avenue Inn Bed and Breakfast", "Shafer Ballille Mansion Bed
& Breakfast", "Ottendun House Bed and Breakfast", "Gallight
Inn", "Bed and Breakfast Inn Seattle", "Silver Cloud Hotel -
Seattle Broadway", "Hyatt House Seattle", "Mozart Guest House",
"Quality Inn & Suites Seattle Center", "MarQueen Hotel"

查找和指定酒店相似度最高的Top10家酒店

实现步骤:

- 数据清洗 (统一小写、去除停用词)
- 使用TF-IDF提取n-gram特征
- 计算余弦相似度矩阵
- 根据用户选择的酒店返回相似推荐

效果验证: 推荐结果与谷歌搜索结果吻合度较高, 验证了方法的有效性

代码关键点

```

176 # 定义模型
177 def model_builder(input_shape):
178     # 创建卷积神经网络模型
179     model = Sequential()
180     # 添加卷积层
181     model.add(Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape=input_shape))
182     # 添加池化层
183     model.add(MaxPooling2D(pool_size=(2, 2)))
184     # 添加全连接层
185     model.add(Flatten())
186     model.add(Dense(100, activation='relu'))
187     model.add(Dense(10, activation='softmax'))
188     # 编译模型
189     model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
190     return model
191
192 # 训练模型
193 def train_model(model, train_data_loader, validation_data_loader):
194     # 训练模型
195     model.fit_generator(train_data_loader.get_data_generator(),
196                       steps_per_epoch=train_data_loader.steps_per_epoch,
197                       validation_data=validation_data_loader.get_data_generator(),
198                       validation_steps=validation_data_loader.steps_per_epoch,
199                       epochs=10)
200
201 # 评估模型
202 def evaluate_model(model, validation_data_loader):
203     # 评估模型
204     score, _ = model.evaluate(validation_data_loader.get_data_generator(),
205                             steps=validation_data_loader.steps_per_epoch)
206     return score
207
208 # 主函数
209 def main():
210     # 加载数据
211     train_data_loader = DataGenerator(train_data_path)
212     validation_data_loader = DataGenerator(validation_data_path)
213     # 构建模型
214     model = model_builder(train_data_loader.get_data_generator().get_shape()[1:])
215     # 训练模型
216     train_model(model, train_data_loader, validation_data_loader)
217     # 评估模型
218     score = evaluate_model(model, validation_data_loader)
219     print('Model accuracy: %f' % score)
220
221 if __name__ == '__main__':
222     main()

```

CountVectorizer: 用于n-gram特征提取

TfidfVectorizer: 实现TF-IDF特征转换

linear_kernel: 高效计算余弦相似度

pandas.Series: 用于相似度排序和索引处理

5) 为酒店建立内容推荐系统 47:23

- 基于内容的推荐
 - 文本特征提取

[illegible]

文本预处理: 使用clean_text函数将文本全部转为小写, 消除大小写差异

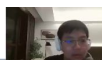
N-Gram特征：提取1-3元语法组合作为特征，共生成26000多维特征向量

TF-IDF计算:

- **TF(词频)**: $TF = \frac{\text{单词出现次数}}{\text{文档总单词数}}$, 反映单词在文档中的重要性
- **IDF(逆文档频率)**: $IDF = \log(\frac{\text{文档总数}}{\text{单词出现的文档数} + 1})$, 衡量单词区分度
- 最终特征值为 $TF \times IDF$, 未出现的单词特征值为0

相似度计算

为酒店建立内容推荐系统



TF-IDF:

• TF: Term Frequency, 词频 $TF = \frac{\text{单词次数}}{\text{文档中总单词数}}$

一个单词的重要性和它在文档中出现的次数成正比。

• IDF: Inverse Document Frequency, 逆向文档频率

一个单词在文档中的区分度。这个单词出现的文档数越少, 区分

度越大, IDF越大 $IDF = \log \frac{\text{文档总数}}{\text{单词出现的文档数} + 1}$



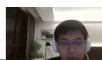
■ **稀疏性问题:** 26000维特征中大部分为0, 矩阵非常稀疏

■ **相似度度量:** 使用线性核函数计算余弦相似度, 公式为

■ **结果解释:** 相似度范围0-1, 1表示完全相同, 0表示完全不相关

○ 推荐算法实现

为酒店建立内容推荐系统



基于相似度和指定酒店的名称, 返回TOP10酒店

def recommendations(name, cosine_similarities):

recommended_hotels = []

为相似矩阵添加名称索引

idx = indices(indices == name).index[0]

print(idx)

对1-idx范围内的相似度和分数按照从大到小进行排序

score_series = pd.Series(cosine_similarities[idx].sort_values(ascending=False))

获取相似度最大的10个 (除了自己)

top_10_indices = score_series.nlargest(10).index

获取推荐列表

for i in top_10_indices:

recommended_hotels.append(score_series[i])

return recommended_hotels

print(recommendations('Hilton Seattle Airport & Conference Center'))

print(recommendations('The B & B Hotel'))

```
idx=40
1 Embassy Suites by Hilton Seattle Tacoma International Airport,
DoubleTree by Hilton Hotel Seattle Airport, Seattle Airport
Marriott, Hotel & Seattle Sea Inn Airport South, Yoma Lodge
Seattle Airport North, Four Points by Sheraton Downtown Seattle
Center, Knights Inn Seattle, Yoma Lodge Downtown Bellevue,
Hampton Inn Seattle/Southcenter, Radisson Hotel Seattle
Airport
idx=116
1210 Ramona Inn Bed and Breakfast, Sheraton Bellevue Hotel Bed
& Breakfast, Chittenden House Bed and Breakfast, Gaslight
Cafe, Bed and Breakfast Inn Seattle, SilverLined Hotel,
Seattle Broadway, Hyatt House Seattle, Mount Gast House,
Quality Inn & Suites Seattle Center, RamQueen Hotel
```

查找和指定酒店相似度最高的Top10家酒店

■ **推荐逻辑:**

- 根据酒店名称找到对应索引
- 获取该酒店的相似度向量并降序排序
- 排除自身(相似度1.0)后取Top10

■ **代码关键点:**

- 使用pd.Series().sort_values(ascending=False)进行排序
- 通过iloc[1:11]跳过自身取前10个
- 示例输出显示机场酒店与B&B酒店各自形成聚类

○ 算法优化思考

为酒店建立内容推荐系统

基于内容的推荐:

- Step1, 对酒店描述 (Desc) 进行特征提取
 - N-Gram, 提取N个连续字的集合, 作为特征
 - TF-IDF, 按照(min, df, max, df)提取关键词, 并生成TFIDF矩阵
- Step2, 计算酒店之间的相似度矩阵
 - 余弦相似度
- Step3, 对于指定的酒店, 选择相似度最大的Top-K个酒店进行输出

Thinking: N-Gram + TF-IDF的特征表达会让特征矩阵非常系数, 计算量大, 有没有更合适的方式?

■ **传统方法局限:**

- 高维稀疏矩阵计算效率低
- 152家酒店即产生26000维, 扩展性差

■ **Embedding替代方案:**

- 将特征降维为固定大小向量(如300维)
- 解决稀疏性问题, 提升计算效率
- 任何对象均可向量化后进行相似度计算

○ 核心流程总结

■ **三步流程:**

Word Embedding

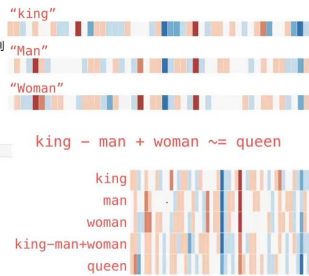


将Word进行Embedding:

- 我们将King与其他单词进行比较, 可以看到Man和Woman更相近
 - 同样有了向量, 我们还可以进行运算
- king-man+woman与queen的相似度最高

```
model.most_similar(positive=["king","woman"], negative=["man"])
```

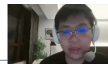
```
{('queen', 0.852360387770834),
 ('throne', 0.766235381534836),
 ('prince', 0.7592144012811372),
 ('daughter', 0.747880320398797),
 ('elizabeth', 0.746023980070894),
 ('princess', 0.742457032050743),
 ('kingdom', 0.732741182074585),
 ('monarch', 0.721448077223643),
 ('queen', 0.718480248648689),
 ('widow', 0.709943056100874)}
```



-
- 向量关系:
 - "Man"和"Woman"向量更相近
 - 向量运算示例: $\text{king} - \text{man} + \text{woman} \approx \text{queen}$
- 相似度验证:
 - 计算结果与"queen"相似度最高 (0.852)
 - 其他相关词: "throne"(0.766),"prince"(0.759)等
- Embedding的定义与特性 59:03

Word Embedding

1.2.8

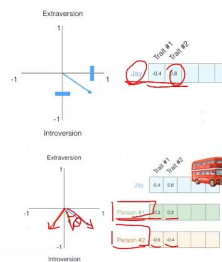


什么是Embedding:

- 一种降维方式, 将不同特征转换为维度相同的向量
- 高维变量转换为one-hot => 维度非常高, 可以把它转换为固定size的embedding向量
- 任何物体, 都可以将它转换为向量的形式, 从Trait #1到#N
- 向量之间, 可以使用相似度进行计算
- 当我们进行推荐的时候, 可以选择相似度最大的

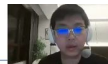
```
cosine_similarity(jay, [0.4, 0.8]) = 0.87 ✓
```

```
cosine_similarity(jay, [0.4, 0.8, -0.5, 0.4]) = -0.28
```



-
- 核心定义: 将不同特征转换为相同维度的向量表示
- 转换优势: 解决one-hot编码维度爆炸问题
- 计算特性: 向量间可通过相似度计算建立关联关系
- 例题2:Word Embedding 59:13

Word Embedding

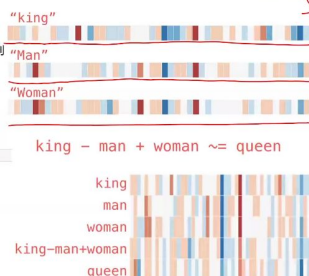


将Word进行Embedding:

- 我们将King与其他单词进行比较, 可以看到Man和Woman更相近
 - 同样有了向量, 我们还可以进行运算
- king-man+woman与queen的相似度最高

```
model.most_similar(positive=["king","woman"], negative=["man"])
```

```
{('queen', 0.852360387770834),
 ('throne', 0.766235381534836),
 ('prince', 0.7592144012811372),
 ('daughter', 0.747880320398797),
 ('elizabeth', 0.746023980070894),
 ('princess', 0.742457032050743),
 ('kingdom', 0.732741182074585),
 ('monarch', 0.721448077223643),
 ('queen', 0.718480248648689),
 ('widow', 0.709943056100874)}
```



-
- 运算验证: 再次演示 $\text{king} - \text{man} + \text{woman} \approx \text{queen}$ 的向量运算
- 相似度排序: 列出与运算结果最相似的前10个单词及其相似度
- Embedding的向量运算 59:25
 - 数学基础: 向量对位加减法保持维度不变 (如50维运算后仍为50维)
 - 物理意义: 虽然计算机不理解语义, 但向量运算结果与人类认知一致
 - 典型示例: $\text{king}(\text{国王}) - \text{man}(\text{男性}) + \text{woman}(\text{女性}) \approx \text{queen}(\text{女王})$ 的语义关系
- Embedding的维度设置 01:01:26
 - 可配置性: 维度大小可自定义 (如50/100/200维)
 - 工具支持: 现有工具支持用户自定义维度进行向量化

- **维度特性**：具体维度不代表特定语义，仅是数学空间的压缩表示
- **Embedding的数学特性 01:02:00**
 - **抽象本质**：对计算机而言仅是数字向量，不关心具体语义
 - **应用原理**：利用向量空间中的数学关系模拟语义关系
 - **神奇之处**：纯数学运算能产生符合人类认知的结果，这是AI强大表现的基础

7) Word2Vec 01:04:01

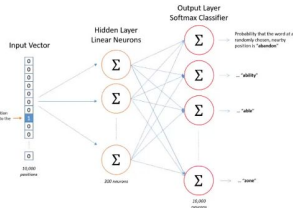
- **Word2Vec的定义**

Word Embedding



Word2Vec:

- 通过Embedding，把原先词所在空间映射到一个新的空间中，使得语义上相似的单词在该空间内距离相近。
- Word Embedding => 学习隐藏层的权重矩阵
- 输入是one-hot编码
- 隐藏层的神经元数量为hidden_size (Embedding Size)
- 对于输入层和隐藏层之间的权重矩阵W，大小为 [vocab_size, hidden_size]
- 输出层为[vocab_size]大小的向量，每一个值代表着输出一个词的概率



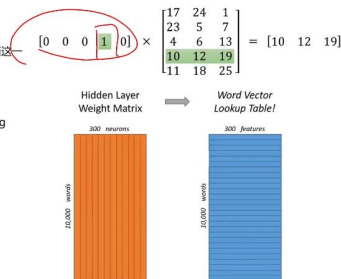
-
- **核心思想**：通过Embedding将词从原始空间映射到新空间，使语义相似的词在新空间中距离相近
- **输入输出**：
 - 输入：**one-hot编码**（如10000维向量，仅1个位置为1）
 - 输出：**[vocab_size]**大小的概率向量，表示预测词的概率分布
- **网络结构**：
 - 隐藏层神经元数=Embedding Size（如300）
 - 权重矩阵W大小：**[vocab_size, hidden_size]**
- **Word Embedding到Word2Vec的过程 01:04:31**

Word Embedding



对于输入的one-hot编码：

- 在矩阵相乘的时候，选取出行中的某一行，而这一行就是输入词语的word2vec表示
- 隐藏层的节点个数 = 词向量的维数
- 隐藏层的输出是每个输入单词的Word Embedding
- word2vec，实际上就是一个查找表



-
- **压缩原理**：
 - **原始问题**：one-hot编码过于稀疏（如10000维中仅1个1）
 - **解决方案**：通过**矩阵乘法**将**高维向量压缩到低维**（如300维）
 - **数学实现**：one-hot向量×权重矩阵W相当于查表操作
- **训练目标**：
 - **自监督任务**：预测上下文邻居词（如扣掉中心词让模型预测）
 - **优化目标**：使预测概率分布与实际上下文分布一致
- **价值体现**：
 - **存储效率**：300维向量可保留99%信息量
 - **语义保持**：压缩后相似词向量距离相近
- **Word2Vec的两种模式 01:10:58**

Word Embedding



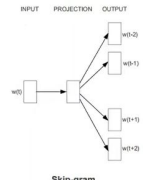
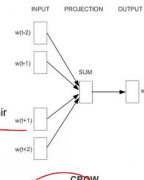
Word2Vec的两种模式:

- Skip-Gram, 给定input word预测上下文

$P(\text{he} | \text{rests})$ $P(\text{in} | \text{rests})$
 $P(\text{life} | \text{rests})$ $P(\text{peace} | \text{rests})$
... Beret of life he **rests** in peace! If you hadn't nailed hir

- CBOW, 给定上下文, 预测input word (与 Skip-Gram相反)

... Beret of life he **rests** in peace! If you hadn't nailed him ...
 $P(\text{rests} | [\text{he}, \text{in}, \text{life}, \text{peace}]))$



-
- **Skip-Gram:**
 - 输入: 中心词
 - 输出: 预测上下文词的概率分布
 - 示例: 给定"rests", 预测["he","in","life","peace"]的概率
- **CBOW:**
 - 输入: 上下文词
 - 输出: 预测中心词的概率
 - 示例: 给定["he","in","life","peace"], 预测"rests"的概率
- 对比: 两种模式输入输出相反, **Skip-Gram**更适合处理低频词
- **Word2Vec工具 01:11:44 Gensim**

-
- **关键参数:**
 - size: 向量维度 (默认100)
 - window: 上下文窗口大小
 - min_count: 词频阈值 (默认5)
 - workers: 并行线程数
- **操作接口:**
 - 模型保存: `model.save(fname)`
 - 相似度计算: `model.wv.similarity()`
 - 向量运算: `model.wv.most_similar(positive=[], negative=[])`
- 例题: 西游记人物相似度计算 01:11:54

实现步骤:

- 使用jieba分词处理原文
- 构建sentence迭代器
- 训练Word2Vec模型（设置size=100/128）
- 计算词向量相似度
- **典型结果：**
 - 孙悟空-猪八戒相似度：0.939
 - 孙悟空-孙行者相似度：0.983
 - 向量运算：孙悟空+唐僧-孙行者≈"菩萨"(0.954)
- Word Embedding总结 01:19:17

Summary



Word2Vec工具的使用：

- Word Embedding就是将Word嵌入到一个数学空间里，Word2vec，就是词嵌入的一种
 - 可以将sentence中的word转换为固定大小的向量表达（Vector Representations），
 - 其中意义相近的词将被映射到向量空间中相近的位置。
 - 将待解决的问题转换为单词word和文章doc的对应关系
- 大V推荐中，大V => 单词，将每一个用户关注大V的顺序 => 文章
商品推荐中，商品 => 单词，用户对商品的行为顺序 => 文章

- **核心价值：**
 - 将离散符号转为连续向量，保留语义关系
 - 支持向量运算（如king-man+woman≈queen）
- **应用扩展：**
 - 用户行为建模：用户关注序列⇒文档
 - 商品推荐：用户行为序列⇒文档
- **实现本质：**
 - 通过神经网络学习压缩表示
 - 保持原始上下文预测任务的准确性

二、知识讲解01:23:30

1. 例题:三国演义嵌入 01:23:46

打卡：三国演义embedding

使用Gensim中的Word2Vec对三国演义进行Word Embedding

- 分析和曹操最相近的词有哪些
- 曹操+刘备-张飞=?

数据集：three_kingdoms.txt

- **实践目标：**使用Gensim中的Word2Vec对《三国演义》进行词向量训练
- **分析任务：**
 - 找出与"曹操"语义最相近的词语
 - 完成向量运算：曹操+刘备-张飞=?
- **数据集：**three_kingdoms.txt（约27万字）
- **技术原理：**通过词向量将文本特征压缩到低维空间（如100维），保留语义关系

2. Word2Vec工具的使用 01:24:45

1) Word2Vec工具简介 01:24:48

Word2Vec工具



```
# 将Word转换成Vec, 然后计算相似度
from gensim.models import word2vec
import multiprocessing

# 如果目录中有多个文件, 可以使用PathLineSentences
sentences = word2vec.PathLineSentences('./segment')

# 设置模型参数, 进行训练
model = word2vec.Word2Vec(sentences, size=100, window=3, min_count=1)
print(model.wv.similarity('孙悟空', '猪八戒'))
print(model.wv.similarity('孙悟空', '孙行者'))

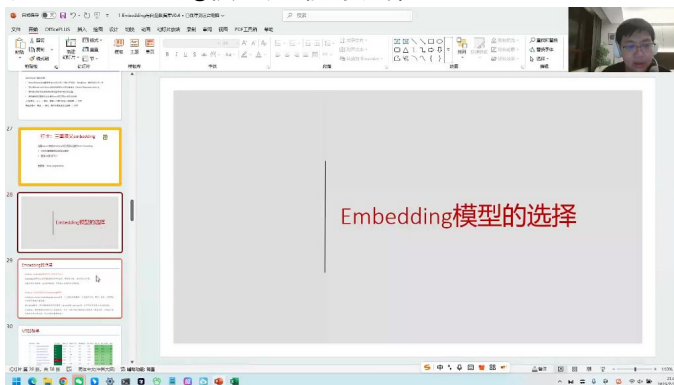
# 设置模型参数, 进行训练
model2 = word2vec.Word2Vec(sentences, size=128, window=5, min_count=5,
workers=multiprocessing.cpu_count())

# 保存模型
model2.save('./models/word2vec.model')
print(model2.wv.similarity('孙悟空', '猪八戒'))
print(model2.wv.similarity('孙悟空', '孙行者'))
print(model2.wv.most_similar(positive=['孙悟空', '唐僧'], negative=['孙行者']))
```

```
0.56224105
0.91238627
0.9133349
0.9293375
['曹操', 0.9546594619750977], ('周瑜', 0.9491345286369324),
('刘备', 0.943795688997559), ('诸葛亮', 0.943665623664855),
('周瑜', 0.9424783786082317), ('曹操', 0.9399656464822764),
('刘备', 0.937991202877594), ('太子', 0.9359638698946486), ('众',
0.933920006854826), ('孙权', 0.9319132363189697)]
```

-
- **核心功能:**
 - 将词语转换为固定维度的向量 (如100维或128维)
 - 计算词语间的余弦相似度 (0-1范围, 越接近1越相似)
- **参数设置:**
 - size: 向量维度 (典型值100/128/256)
 - window: 上下文窗口大小 (常用3-5)
 - min_count: 词频阈值 (过滤低频词)
- **典型应用:**

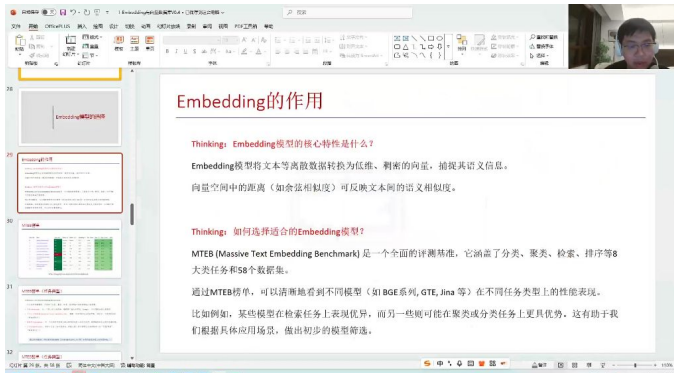
2) 例题2: Embedding模型压缩的规律 01:27:30



-
- **压缩原理:**
 - 将高维稀疏特征 (如one-hot编码) 转换为低维稠密向量
 - 典型压缩比例: 1万维→300维 (可根据需求调整)
- **维度选择:**
 - 常用2的幂次数: 128/256/512/1024
 - 小规模语料可适当降低维度 (如50-100维)
- **与大模型区别:**
 - **Embedding模型:** 特征提取器, 输出固定维度向量
 - **大语言模型:** 生成式模型, 处理复杂语义任务
- **模型选择依据:**
 - 参考MTEB评测基准 (涵盖8类任务58个数据集)
 - 不同模型在不同任务表现各异 (如BGE擅长检索, GTE长于分类)

3. Embedding的作用 01:27:48

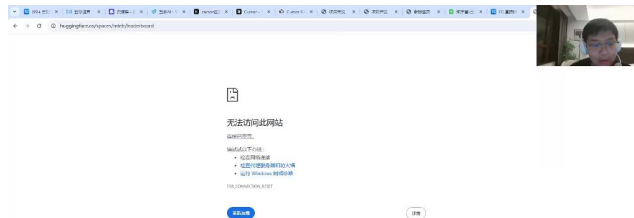
1) 嵌入模型选择 01:28:36



-
- **本质特征**：Embedding模型是**词压缩模型**，通过将离散数据转换为低维稠密向量来捕捉语义特征，而非生成式模型
- **训练方法**：使用watch wack工具训练，通过大量语料迭代学习**矩阵参数**（初始随机→逐步优化）
- **典型应用**：适用于分类、检索、排序等特征抽取任务，不能直接生成内容
- **性能差异**：不同模型在语义理解精度上有显著差异，需根据下游任务选择

2) MTEB榜单 01:29:46

- MTEB介绍 01:29:49



-
- **基准构成**：涵盖8大类任务和58个数据集，包括分类/聚类/检索/排序等
- **模型对比**：可横向比较BGE/GTE/Jina等系列模型在不同任务的表现
- **使用方式**：通过huggingface平台访问MTEB榜单（需特殊网络环境）
- MTEB榜单（任务类型） 01:32:04

MTEB榜单（任务类型）

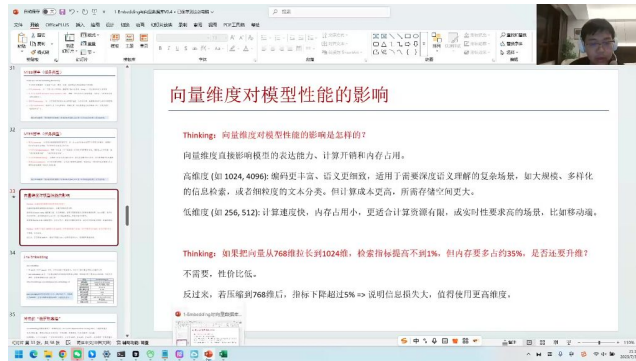
MTEB (Massive Text Embedding Benchmark) :

一个全面的评测基准，它涵盖了分类、聚类、检索、排序等8大类任务和58个数据集。

- **检索 (Retrieval)**：从一个庞大的文档库中，根据用户输入的查询 (Query)，找出最相关的文档列表。
- **语义文本相似度 (Semantic Textual Similarity, STS)**：判断一对句子的语义相似程度，并给出一个连续的分值（例如1到5分）。
- **重排序 (Reranking)**：对一个已经初步检索出的文档列表进行二次优化排序，使得最相关的文档排在最前面。
- **分类 (Classification)**：将单个文本（如电影评论、新闻文章）划分到预定义类别中（如“正面/负面”、“体育/科技”）。

通过MTEB榜单，可以看到不同模型（如BGE系列, GTE, Jina等）在不同任务类型上的性能表现。

-
- **检索任务(Retrieval)**：从文档库中找出与查询最相关的文档列表
- **语义相似度(STS)**：评估句子对的语义**相似程度**（1-5分连续评分）
- **重排序(Reranking)**：对初步检索结果进行优化排序
- **分类任务**：将文本划分到预定义类别（如情感分析）
- **聚类任务**：无监督地将相似文本自动分组
- **双语挖掘**：跨语言语义匹配
- **向量维度对模型性能的影响** 01:35:58



- 高维度优势: 1024/4096维能编码更丰富语义, 适合复杂场景(如细粒度分类)
- 低维度优势: 256/512维计算速度快, 适合实时性要求高的场景(如移动端)
- 选择标准: 维度提升1%性能但增加35%内存时不建议升维; 降维导致>5%性能损失时应保持高维
- 典型案例: Qwen3模型有不同维度版本(0.6B/4B/8B), 4096维版本效果最优但成本最高
- 相关问题解答 01:36:18

Rank	Model	Zero-shot	Clustering	Instruction R.	Multilabel class.	Pair Classification	Summing	Retrieval
1	gpt4o	99.99	99.99	99.99	99.99	99.99	99.99	99.99
2	gpt4o-mini	99.99	99.99	99.99	99.99	99.99	99.99	99.99
3	gpt4o-mini-2024-07-18	99.99	99.99	99.99	99.99	99.99	99.99	99.99
4	gpt4o-mini-2024-07-18	99.99	99.99	99.99	99.99	99.99	99.99	99.99
5	gpt4o-mini-2024-07-18	99.99	99.99	99.99	99.99	99.99	99.99	99.99
6	gpt4o-mini-2024-07-18	99.99	99.99	99.99	99.99	99.99	99.99	99.99
7	gpt4o-mini-2024-07-18	99.99	99.99	99.99	99.99	99.99	99.99	99.99
8	gpt4o-mini-2024-07-18	99.99	99.99	99.99	99.99	99.99	99.99	99.99
9	gpt4o-mini-2024-07-18	99.99	99.99	99.99	99.99	99.99	99.99	99.99
10	gpt4o-mini-2024-07-18	99.99	99.99	99.99	99.99	99.99	99.99	99.99

- 矩阵学习: Embedding矩阵通过数百次迭代训练获得, 非人工设定
- 分词模型关系: 与token分词模型无绑定关系, 两者训练目标和参数不共享
- 视频处理: 视频Embedding常用CLIP或DINO模型
- 逻辑体现: 50维即可体现基础逻辑, 更高维度(如1000维)能提升精细度
- Embedding和LLM模型的区别 01:39:08

Rank	Model	Zero-shot	Clustering	Instruction R.	Multilabel class.	Pair Classification	Summing	Retrieval
1	gpt4o	99.99	99.99	99.99	99.99	99.99	99.99	99.99
2	gpt4o-mini	99.99	99.99	99.99	99.99	99.99	99.99	99.99
3	gpt4o-mini-2024-07-18	99.99	99.99	99.99	99.99	99.99	99.99	99.99
4	gpt4o-mini-2024-07-18	99.99	99.99	99.99	99.99	99.99	99.99	99.99
5	gpt4o-mini-2024-07-18	99.99	99.99	99.99	99.99	99.99	99.99	99.99
6	gpt4o-mini-2024-07-18	99.99	99.99	99.99	99.99	99.99	99.99	99.99
7	gpt4o-mini-2024-07-18	99.99	99.99	99.99	99.99	99.99	99.99	99.99
8	gpt4o-mini-2024-07-18	99.99	99.99	99.99	99.99	99.99	99.99	99.99
9	gpt4o-mini-2024-07-18	99.99	99.99	99.99	99.99	99.99	99.99	99.99
10	gpt4o-mini-2024-07-18	99.99	99.99	99.99	99.99	99.99	99.99	99.99

- 核心差异:
 - 任务类型: Embedding是语义理解型AI, LLM是生成型AI
 - 成本对比: deepseek-r1 671B模型成本比4B/8B模型高100倍
 - 响应速度: Embedding处理约100ms, LLM同任务需50s以上
- 应用场景:
 - Embedding适合知识库检索等特征抽取任务
 - LLM适合需要内容生成的复杂任务
- 性能平衡: LLM理解精度可能高5-10%, 但时间成本呈指数级增长

3) 向量维度对模型性能的影响 01:41:32

向量维度对模型性能的影响



Thinking: 向量维度对模型性能的影响是怎样的?

向量维度直接影响模型的表达能力、计算开销和内存占用。

高维度 (如 1024, 4096): 编码更丰富、语义更细致, 适用于需要深度语义理解的复杂场景, 如大规模、多样化的信息检索, 或者细粒度的文本分类。但计算成本更高, 所需存储空间更大。

低维度 (如 256, 512): 计算速度快, 内存占用小, 更适合计算资源有限, 或实时性要求高的场景, 比如移动端。

Thinking: 如果把向量从768维拉长到1024维, 检索指标提高不到1%, 但内存要多占约35%, 是否还要升维?

- 维度与性能关系: 向量维度直接影响模型的表达能力、计算开销和内存占用
- 高维度特点:
 - 典型维度: 1024/4096维
 - 优势: 编码更丰富、语义更细致
 - 适用场景: 大规模信息检索、细粒度文本分类等复杂任务
 - 代价: 计算成本高, 存储空间需求大 (如768维→1024维内存增加35%)
- 低维度特点:
 - 典型维度: 256/512维
 - 优势: 计算速度快, 内存占用小
 - 适用场景: 移动端应用、实时性要求高的系统
- 性价比考量: 升维决策需权衡指标提升幅度 (如检索指标提高<1%) 与资源消耗增加比例

4) 神奇的俄罗斯套娃 01:43:48

- 俄罗斯套娃模型: Matryoshka Representation Learning (MRL)

神奇的“俄罗斯套娃”



Jina-embeddings训练时使用了一种特殊技术 (Matryoshka Representation Learning, MRL) => 俄罗斯套娃

生成完整向量: 模型总是先在内部生成一个最完整、维度最高 (比如 2048维) 的向量。

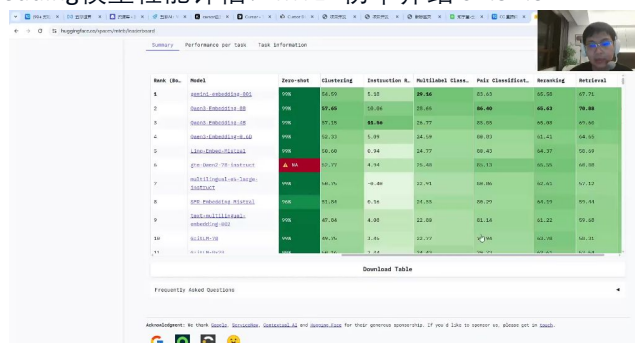
按需截断: 这个长向量有一个非常神奇的特性, 它的前128维、前256维、前512维.....本身就是一组高质量的、独立的、可以正常使用的短向量。



用户指定: 当调用模型时, 可以通过 `embedding_size` 告诉模型 “我这次只需要前512维就够了” => 模型就会截断向量, 只返回前512维

- 核心机制:
 - 先生成完整高维向量 (如2048维)
 - 支持按需截取前N维 (128/256/512维等)
- 技术优势:
 - 各维度段均可独立使用
 - 截断后质量损失小 (前128维仍保持高质量)
- 实现方式: 通过 `embedding_size` 参数指定所需维度
- 套娃模型的应用场景 01:44:54
 - 社交媒体场景:
 - 推荐使用: 128维向量
 - 优势: 实现实时评论检索 (用户问题与评论均用128维)
 - 金融分析场景:
 - 案例: 私募基金投研报告分析
 - 推荐使用: 高精度维度 (日耗约7000美金)

- 价值：精准匹配百万份文档，支撑投资决策
- Embedding模型作为大模型的辅助工具 01:46:35
 - 角色定位：
 - 功能：语义理解与信息过滤
 - 优势：突破大模型上下文窗口限制
 - 工作流程：
 - 向量化处理语料
 - 相似度计算筛选
 - 将精华片段输入大模型
 - 典型差异：
 - 大模型：生成内容（如写报告）
 - Embedding模型：理解内容（相似度计算）
- 模型尺寸的选择：任务价值与需求 01:47:55
 - 决策要素：
 - 任务价值：高价值任务倾向高维（如研报报告可接受周级产出）
 - 实时性要求：实时系统选择低维
 - 典型案例：
 - 快速响应：选择128维（Jina模型截断）
 - 深度分析：使用完整维度（如2000维）
- 单语言与多语言模型的选择 01:48:31
 - 单语言模型：
 - 典型代表：bge-large-ch（中文专用）
 - 优势：本土化表达精准（如"七天无理由退货"）
 - 适用场景：单一语言服务（如中文客服系统）
 - 多语言模型：
 - 典型场景：跨国连锁酒店评论分析
 - 优势：跨语言语义对齐（clean room≈干净房间）
 - 缺点：单语种精度稍逊
- 多语言模型：应用场景与优缺点 01:49:35
 - 实现原理：
 - 多语言联合训练
 - 共享向量空间映射
 - 典型应用：
 - 跨语言检索（中文查询匹配日文评论）
 - 多语种知识库构建
- Embedding模型性能评估：MTEB榜单介绍 01:51:31



The screenshot displays the MTEB leaderboard with the following data:

Rank (by...)	Model	Zero-shot	Clustering	Instruction R.	Multilabel class.	Pair Classification	Recomm.	Retrieval
1	openai-embedding-ada-002	99%	94.59	5.10	89.56	89.63	94.58	87.76
2	openai-embedding-ada-001	99%	93.48	10.84	75.45	86.40	88.43	79.88
3	openai-embedding-ada-002	99%	91.75	85.84	79.77	85.95	94.98	85.85
4	openai-embedding-ada-001	99%	90.73	5.99	74.19	88.09	92.41	84.93
5	openai-embedding-ada-002	99%	89.00	6.94	74.77	85.47	94.79	79.19
6	openai-embedding-ada-001	99%	87.77	4.94	75.48	85.17	94.58	84.98
7	openai-embedding-ada-002	99%	86.75	4.40	72.11	88.98	92.45	81.12
8	openai-embedding-ada-001	99%	81.74	6.34	74.16	88.29	94.79	89.14
9	openai-embedding-ada-002	99%	87.14	8.00	72.09	85.14	93.22	85.85
10	openai-embedding-ada-001	99%	86.75	5.01	72.17	87.18	94.79	84.15
11	openai-embedding-ada-002	99%	86.75	5.01	72.17	87.18	94.79	84.15

- 榜单特点：
 - 覆盖100+模型、1000+语言
 - 包含131个任务、9种任务类型

- 使用注意：
 - 模型表现存在任务差异（如检索vs分类）
 - 仅作参考（测试集可能与业务场景不符）
- 模型选型的关键步骤 01:52:29

如何选择适合的Embedding模型



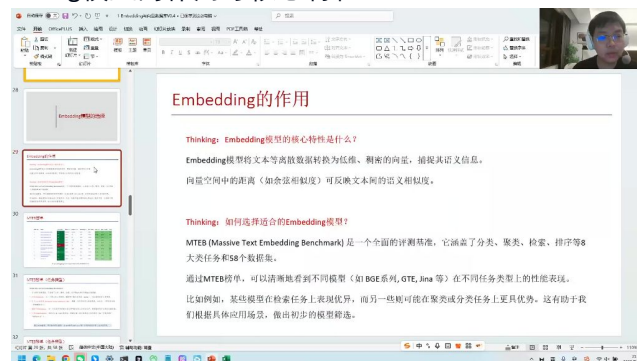
Thinking: 如何选择适合的Embedding模型，依靠MTEB就可以？

模型选型是一个系统的过程，不能仅仅依赖于公开榜单。包括以下关键步骤：

- 明确业务场景与评估指标：首先定义核心任务是检索、分类还是聚类？并确定衡量业务成功的关键指标，如检索召回率 (Recall@K)、准确率 (Accuracy) 或 NDCG。
- 构建“黄金”测试集：准备一套能真实反映您业务场景和数据分布的高质量小规模测试集。比如，构建一系列“问题-标准答案”对 => 评估模型好坏的“金标准”。
- 小范围对比测试 (Benchmark)：从MTEB榜单中挑选几款排名靠前且符合需求（如语言、维度）的候选模型。使用“黄金”测试集，对这些模型进行评测。

Embedding模型的选择属于综合评估，即结合测试结果、模型的推理速度、部署成本 => 做出最终决策

- 四步法：
 - 明确业务指标 (Recall@K/Accuracy/NDCG)
 - 构建黄金测试集 (100+高质量样本)
 - 小范围对比测试 (结合MTEB初筛)
 - 综合评估 (质量+速度+成本)
- 核心原则：
 - 拒绝唯榜单论
 - 建立业务专属评估标准
- Embedding模型的作用与核心特性 01:55:28



- 核心功能：
 - 离散数据→稠密向量（维度压缩）
 - 语义相似度计算（余弦相似度）
- 评估体系：
 - MTEB基准：8类任务/58个数据集
 - 典型任务差异：检索vs聚类vs分类

4. 休息 01:55:56

- 1) 休息五分钟 01:56:06
 - 2) 休息结束，准备继续 02:00:52
5. 答疑 02:01:08

- **Embedding模型套娃**：可以嵌套使用多个独立的embedding模型，512维度的向量也是可行的特殊技术方案
- **向量与原文转换**：通过向量数据库同时存储原文和向量化表示，实现双向转换管理

6. Embedding与向量数据库

- 1) Embedding核心作用

Rank	Model	Zero-shot	Few-shot	Number of R.	Embedding D.	Raw Tokens	Mean F1	Mean Recall	F1%
1	gpt4o-mini-2024-04-09	97.5	97.5	Unknown	3872	2545	88.37	90.45	89.4
2	Qwen3-Embedding-4B	97.5	97.5	78	Unknown	37140	88.38	88.45	88.4
3	Qwen3-Embedding-4B	97.5	97.5	2545	48	2545	88.38	88.45	88.4
4	Qwen3-Embedding-4B	97.5	97.5	2545	2545	2545	88.38	88.45	88.4
5	Llama-3.3-70B-Instruct	97.5	97.5	78	Unknown	37140	88.38	88.45	88.4
6	gpt4o-mini-2024-04-09	97.5	97.5	2545	78	2545	88.38	88.45	88.4
7	Qwen3-Embedding-4B	97.5	97.5	2545	2545	2545	88.38	88.45	88.4
8	Qwen3-Embedding-4B	97.5	97.5	2545	2545	2545	88.38	88.45	88.4
9	Qwen3-Embedding-4B	97.5	97.5	2545	2545	2545	88.38	88.45	88.4
10	Qwen3-Embedding-4B	97.5	97.5	2545	2545	2545	88.38	88.45	88.4

https://huggingface.co/spaces/mteb/leaderboard

-
- 语义关系判断: 通过向量化判断上下文语义的关联程度
- 模型性能对比: 不同模型在MTEB榜单上的表现差异显著, 如Qwen3-Embedding-4B在多个任务中表现突出

2) MTEB任务类型

MTEB榜单 (任务类型)

- 聚类 (Clustering): 在没有任何预设标签的情况下, 将一组文本自动地分成若干个有意义的群组, 使得同一组内的文本语义相似, 不同组间的文本语义差异大。
- 对分类 (Pair Classification): 判断一对文本 (句子或段落) 是否具有某种特定关系, 通常是二分类问题, 如“是否是重复问题”、“是否是转述关系”。
- 双语挖掘 (Bilingual Mining): 从两种不同语言的大量句子中, 找出互为翻译的句子对。对于机器翻译至关重要。
- 摘要 (Summarization): 这个任务比较特殊, 它不是让模型生成摘要, 而是评估一个机器生成的摘要与人工撰写的参考摘要之间的语义相似度。

通过MTEB榜单, 可以看到不同模型 (如 BGE 系列, GTE, jina 等) 在不同任务类型上的性能表现。

-
- 聚类任务: 无监督地将语义相似的文本自动分组
- 对分类任务: 判断文本对是否具有特定关系 (如重复问题识别)
- 双语挖掘: 跨语言寻找互为翻译的句子对
- 摘要评估: 量化机器生成摘要与人工参考摘要的语义相似度

3) 向量数据库核心概念

向量数据库

Thinking: 如何让LLM“记住”并利用海量、多样的私有知识?

向量数据库: AI时代的核心记忆体

与传统的数据库不同, 向量数据库用于存储和查询由非结构化数据 (如文本、图片、音视频) 转化而来的高维向量嵌入 (Embeddings)。这些向量在多维空间中的距离代表了原始数据的语义相似度。

因此, 向量数据库的核心能力是高效的相似性检索。

Thinking: 向量数据库的核心价值?

- 为大模型提供长期记忆: 弥补LLM上下文窗口 (Context Window) 长度限制和知识更新延迟的问题。
- 实现私有知识库的问答与搜索: 将企业内部文档、产品信息等转化为向量, 实现基于语义的智能检索。
- 赋能推荐系统、以图搜图等多种应用: 通过计算用户、物品的向量相似度, 提供更精准的推荐。

-
- 与传统数据库区别:
 - 存储内容: 高维向量而非结构化数据
 - 查询方式: 近似匹配 (余弦相似度) 而非精确匹配
 - 返回结果: 语义相关的多个结果而非单条记录
- 核心价值:
 - 扩展LLM记忆: 突破上下文窗口限制
 - 实现语义搜索: 支持私有知识库智能检索
 - 赋能推荐系统: 基于向量相似度的物品推荐

4) 主流向量数据库对比

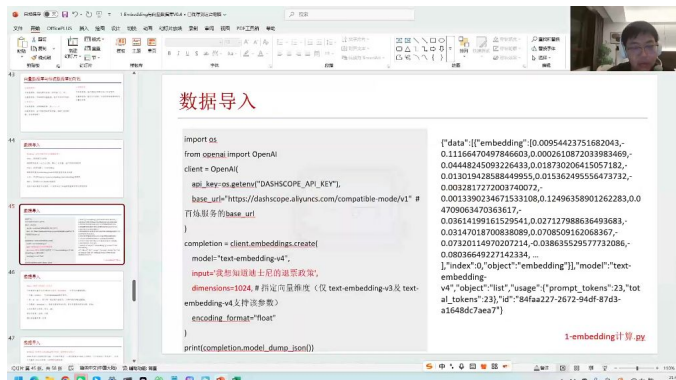
- **Faiss:**
 - 特点: 轻量级专注相似度检索
 - 优势: 查询速度极快
- **Elasticsearch:**
 - 特点: 支持混合检索 (关键词+语义)
 - 优势: 商业场景成熟度高
- **Milvus:**
 - 特点: 基于Go语言的分布式架构
 - 优势: 海量数据分布式处理
- **Pinecone:**
 - 特点: 云原生托管服务
 - 优势: 部署维护简便

5) 向量化技术应用

- **多模态支持:**
 - 文本向量化: 如BERT、GPT等模型
 - 图像向量化: 如ResNet、CLIP等模型
- **特征提取原理:**
 - 通过预训练模型获取抽象特征表示
 - 不同领域需使用专用模型 (如ResNet用于图像分类)

7. 向量数据库 02:02:26

1) 例题: 向量模型与数据库连接 02:11:47



- **Embedding模型核心功能**
 - **语义向量化:** 无论输入文本长度 (从单个词"迪士尼"到2000字文章), 均可通过embedding模型转化为固定维度 (如1024维) 的向量表示
 - **维度压缩特性:** 长文本压缩为低维向量时会产生语义损失, 但保留了核心语义特征
 - **模型兼容性:** 阿里云text-embedding-v4模型支持自定义维度参数 (v3/v4版本特有功能)
- **实践示例**

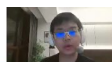
```

1 import os
2 from openai import OpenAI
3 client = OpenAI(api_key=os.getenv("DASHSCOPE_API_KEY"),
4                 base_url="https://dashscope.aliyuncs.com/compatible-mode/v1")
5
6 completion = client.embeddings.create(
7     model='text-embedding-v4',
8     input='我想知道迪士尼的退票政策',
9     dimensions=1024, # 仅v3/v4支持
10    encoding_format="float"
11)
12 print(completion.model_dump_json())

```

- **参数说明：**
 - **dimensions=1024：**指定生成1024维浮点型向量
 - **encoding_format="float"：**确保输出为浮点数格式
- **执行效率：**国内网络调用阿里云API时响应速度较优
- **输出特征：**生成向量包含正负浮点数（如0.00954, -0.11166等），反映多维语义特征
- **扩展问答**
 - **模型训练：**专业领域数据需通过特定语料训练才能生成适配的embedding模型
 - **部署方案：**训练完成的模型可通过标准化接口封装供外部调用
- 2) 有人专业性资料，如何从零开始训练出自己的embedding模型 02:14:48
- **训练工具与模型选择 02:14:53**
 - **Word2Rect工具：**可用于简单训练任务，但精度相对较低，适合基础场景使用
 - **BERT模型应用：**2018年流行的神经网络模型（非大语言模型），可通过语料训练生成特定模型
 - **模型选择原则：**根据任务复杂度选择，简单任务可用Word2Rect，复杂任务建议使用BERT等神经网络
 - **神经网络压缩：**训练后的模型可进行压缩处理，便于后续任务部署和执行
- **分词与向量压缩 02:15:45**
 - **分词过程：**原始句子→分词→得到空格分隔的词语序列（如abcd）
 - **向量压缩：**将分词结果映射到隐空间（如1024维向量），实现对整体语义的理解
 - **分词规则：**中文分词依赖词典（如结巴分词），可自定义添加专有名词词典
 - **Transformer优势：**相比传统方法能更好地处理语义理解任务
- **神经网络模型与推荐系统 02:17:01**
 - **推荐系统架构：**
 - 传统方案：双塔神经网络结构、DeepFM、NFM、Wide&Deep等专用模型
 - 大模型方案：可作为辅助工具，非必需组件
 - **模型特点：**专用模型针对特定数据训练，预测用户偏好，非通用大模型
 - **发展历程：**推荐系统在大模型兴起前已广泛应用神经网络技术
- **神经网络模型与大模型辅助 02:18:13**
 - **大模型辅助功能：**
 - 特征标记：为物品打语义标签
 - 特征抽取：提取物品关键特征
 - 话术生成：解释推荐理由
 - **应用定位：**主要承担特征处理工作，不直接用于推理
 - **工业案例：**制造企业用元数据管理产品型号（如触摸屏型号A/B区分）
- **FAISS库的安装与使用 02:19:03**
 - **安装方式：**pip install faiss（CPU版）或pip install faiss-gpu（GPU版）
 - **核心功能：**高效向量相似度计算库，支持十亿级文档毫秒级检索
 - **版本选择：**根据硬件配置选择对应版本，无GPU设备安装CPU版本即可
- **元数据的定义与管理 02:19:45**
 - **元数据定义：**描述向量的附加信息，包括：
 - 数据来源（文件名/网页/章节）
 - 业务属性（商品类别/型号/价格）
 - 管理信息（创建日期/作者）
 - **工业案例：**制造企业将产品型号作为关键元数据（如触摸屏型号A/B）
 - **存储必要性：**向量化后丢失原始信息，需通过元数据保留关键业务属性
- **元数据管理实现 02:22:48**

数据导入



Step3: 数据与元数据一同导入

将生成的向量及其关联的 **元数据 (Metadata)** 一同存入向量数据库。

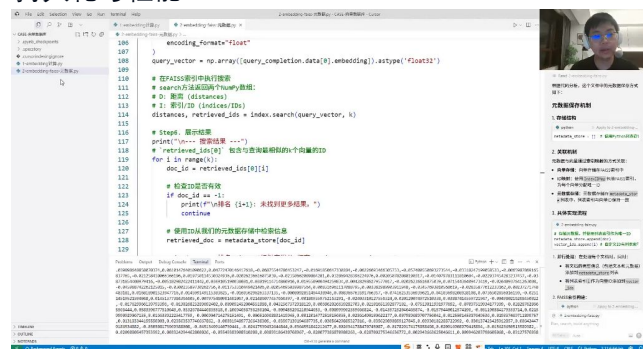
- 向量 (Vector)：生成的Embedding数字数组。
- 唯一ID (ID)：用于唯一标识每个数据点，方便后续的更新或删除。
- 元数据 (Metadata)：描述向量的附加信息，是实现高级检索的关键。例如：

文本来源的文件名、章节、URL

商品的类别、品牌、价格

图片的创建日期、作者

-
- **三要素存储：**
 - 向量：Embedding数字数组
 - 唯一ID：数据点标识符
 - 元数据：实现高级检索的关键
- **实现方式：**使用FAISS的IndexIDMap建立向量与元数据的映射关系
- **类比说明：**类似数据库管理，新增数据自动生成唯一ID
- **代码实现与向量索引 02:24:08**
 - **技术栈：**
 - 使用DashScope的text-embedding-v4模型（兼容OpenAI接口）
 - 生成1024维向量
 - **数据处理流程：**
 - 原始文档准备（含元数据标注）
 - 调用Embedding接口生成向量
 - 构建向量列表和元数据存储
 - **索引优化：**采用精确索引方案，平衡检索精度与速度
- **相似向量检索与结果展示 02:29:20**
 - **检索流程：**
 - 用户问题向量化（需使用相同模型）
 - FAISS的search函数执行K近邻搜索（如K=3）
 - 通过ID关联获取元数据和原始文本
 - **关键参数：**
 - 维度一致性：必须保持1024维
 - 模型一致性：需使用相同embedding模型
 - **结果展示：**输出相似度分数及对应元数据，支持业务决策
- **FAISS持久化与性能 02:31:10**



-
- **持久化能力：**FAISS索引可以保存为.pkl文件，下次直接读取使用，加载速度非常快
- **运行机制：**实际运行时索引数据加载到内存中操作，但支持文件持久化存储
- **性能表现：**官方支持十亿级(e级别)向量检索，典型响应时间在几百毫秒量级
- **检索结果解析**

Embedding与原数据导入Faiss



```
正在为文档生成向量...
- 已处理文档 1/4
- 已处理文档 2/4
- 已处理文档 3/4
- 已处理文档 4/4

FAISS 索引已成功创建，共包含 4 个向量。
正在为查询文本生成向量：“我想了解一下迪士尼门票的退款流程”

--- 搜索结果 ---
--- 排名 1 (相似度得分/距离: 0.3222) ---
ID: 2
原始文本: 对于在线购买的迪士尼门票，如果需要退票，必须在票面日期前48小时通过原购买渠道提交申请，并可能收取手续费。
元数据: {'source': 'online_policy.html', 'category': '退票政策', 'author': 'commerceTeam'}

--- 排名 2 (相似度得分/距离: 0.3312) ---
ID: 0
原始文本: 迪士尼乐园的门票一经售出，原则上不予退换，但在特殊情况下，如恶劣天气导致园区关闭，可在官方指引下选择退票。
元数据: {'source': 'official_faqs_v1.pdf', 'category': '退票政策', 'author': 'Admin'}

--- 排名 3 (相似度得分/距离: 1.0135) ---
ID: 1
原始文本: 购买“奇妙年卡”的用户，可以享受一年内多次入园的特权，并且在餐饮和购物时有折扣。
元数据: {'source': 'annual_pass_rules.docx', 'category': '会员权益', 'author': 'MarketingDept'}
```

2-embedding-faiss-元数据.py

- **距离含义：**返回的相似度得分实际是距离值，距离越大表示相似度越低
- **结果排序：**示例中k=3返回前三个结果，距离分别为0.3222、0.3312、1.0135
- **元数据结构：**包含原始文本和metadata字典（source/category/author等字段）

典型应用流程

■ 数据处理阶段：

- **并行处理：**同时将文档信息存入metadata_store列表，并将列表索引作为向量ID
- **索引构建：**使用IndexFlatL2创建精确索引，通过IndexIDMap封装实现ID映射

■ 查询阶段：

- **向量转换：**将用户问题转换为相同维度的向量
- **快速检索：**FAISS返回相似向量ID和距离值
- **结果关联：**通过ID从metadata_store获取完整信息

元数据管理的更健壮方案 02:36:16

Summary



Thinking: 如何将metadata元数据管理的更健壮？

在更复杂的生产环境中，可以将 metadata_store 这个简单的列表替换为更健壮的存储方案：

- 键值数据库（如 Redis）：通过ID快速查询，性能极高。
- 关系型数据库（如 PostgreSQL）：可以存储更复杂的结构化元数据。
- 文档数据库（如 MongoDB）：非常适合存储JSON格式的元数据。

FAISS 专注于其最擅长的高速向量检索，而元数据的存储和管理可以交给专业的数据库系统负责，实现了架构上的解耦和高效协同。

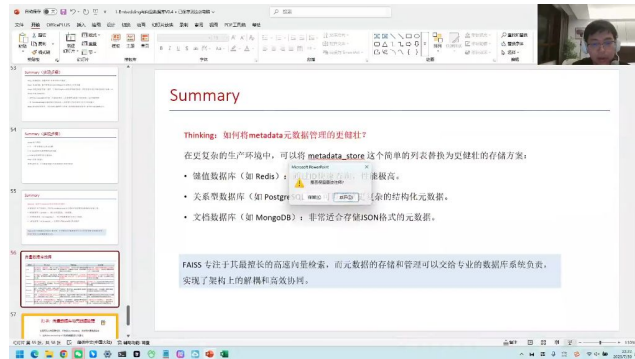
- **基础方案局限：**使用Python列表(metadata_store)临时存储元数据，通过列表索引关联
- **专业数据库方案：**
 - **键值数据库：**如Redis，通过ID快速查询，性能极高
 - **关系型数据库：**如PostgreSQL，适合存储复杂结构化元数据
 - **文档数据库：**如MongoDB，天然支持JSON格式元数据存储
- **架构解耦优势**
 - **职责分离：**FAISS专注向量检索，元数据交给专业数据库管理
 - **实现示例：**
- **扩展性：**支持不同业务场景下的元数据复杂度需求
- **实现细节说明**
 - **ID验证：**检查retrieved_ids是否为-1（无效ID）
 - **结果展示：**格式化输出排名、距离、原始文本和完整元数据
 - **错误处理：**捕获并提示查询过程中的异常情况
- **向量数据库的选择 02:37:44**

向量数据库选择

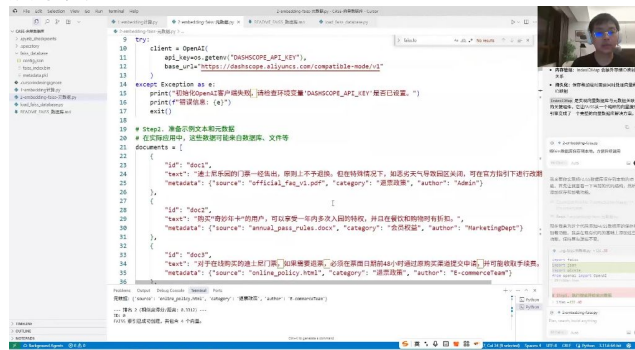


数据库	核心特点	性能表现	适用场景
FAISS	核心算法库，非数据库。由Meta AI开发，提供最广泛、最前沿的ANN索引算法。支持CPU和GPU。	纯粹的性能标杆。在内存中进行向量检索时速度极快，尤其是CPU版本，是衡量其他数据库性能的基准。	算法研究者；需要将向量检索能力深度集成到现有系统中的团队；可以自行构建服务层（API、元数据管理）
Milvus	开源领导者，功能全面。云原生架构，高可扩展。支持多种ANN索引和丰富的调优参数。	在多个公开的ANN基准测试中，尤其是在大规模数据集上表现优异，吞吐量和延迟控制得很好。	需要处理海量数据、对性能和扩展性有要求的企业级应用。适合有一定运维能力的团队进行私有化部署。
Pinecone	全托管的商业先驱。主打"Serverless"和易用性。API设计简洁，专注于提供极致的低延迟检索。	性能非常出色，尤其在低延迟方面有优势。由于是托管服务，其内部优化细节不透明，但用户体验极佳。	追求快速上线。希望将运维负担降至最低的团队。初创公司和需要快速验证AI应用可行性的场景。
Weaviate	开源，内置数据向量化模块。能够连接各种Embedding模型，实现数据的自动向量化，简化了开发流程。	性能良好，易用性强。其自动向量化功能在开发效率上是一大亮点。	希望简化ETL流程，快速构建从数据到向量再到检索的全链路应用的开发者。
Qdrant	开源，以性能和内存安全著称（使用Rust开发）。过滤查询能力强大且高效。	性能强劲，尤其在过滤条件复杂的混合查询场景下表现突出。内存使用效率高。	对检索性能和资源利用率有极致要求的场景。金融、电商等需要复杂过滤规则的应用。
Elasticsearch	通用搜索巨头，扩展向量检索能力。将向量检索（KNN）作为功能之一，可以无缝融合其强大的全文检索、聚合分析能力。	作为一个通用数据库，其向量检索性能通常弱于专门的向量数据库。但在混合搜索（关键词+向量）场景下，表现优异。	业务需求是"以文本搜索为主，向量搜索为辅"的场景。希望在一个统一的平台中解决所有搜索问题，而非追求极致的向量检索性能。

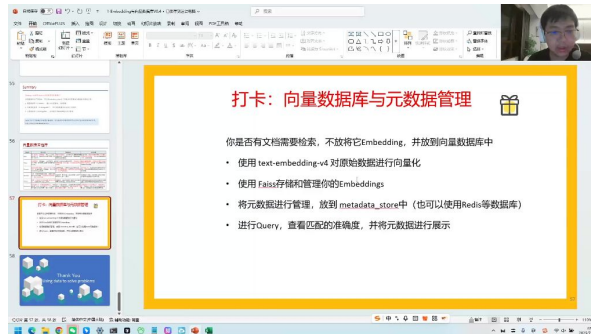
- FAISS
 - 核心定位：Meta AI开发的核心算法库，非完整数据库，是衡量其他数据库性能的基准
 - 性能特点：CPU/GPU版本检索速度极快，纯粹的内存检索性能标杆
 - 适用场景：
 - 算法研究者需要前沿ANN索引算法
 - 需要将向量检索能力深度集成到现有系统的团队
 - 需自行构建服务层（API、元数据管理）的场景
 - 持久化方法：
 - 通过pickle.dump()保存index对象
 - 加载时使用pickle.load()和faiss.read_index()
 - 生成三个文件：config（配置）、bin（索引数据）、metadata（pickle序列化数据）
- Milvus
 - 核心特点：Go语言开发的开源领导者，分布式吞吐能力强
 - 性能表现：在大规模数据集ANN基准测试中表现优异
 - 适用场景：处理海量数据且对性能/扩展性要求高的企业级应用
- Pinecone
 - 架构特点：全托管闭源商业服务，主打Serverless和易用性
 - 性能优势：低延迟检索表现突出（内部优化细节不透明）
 - 适用场景：需要快速上线、最小化运维负担的团队（如初创公司验证AI应用）
- Weaviate
 - 核心功能：内置数据向量化模块，可连接多种Embedding模型
 - 开发优势：自动向量化功能显著提升开发效率
 - 适用场景：需要快速构建从原始数据到向量检索全链路的开发者
- Qdrant
 - 技术特点：Rust开发，以性能和内存安全著称
 - 查询优势：过滤条件复杂的混合查询场景表现突出
 - 适用场景：金融/电商等需要复杂过滤规则的高性能应用
- Elasticsearch
 - 功能定位：通用搜索引擎扩展向量检索能力（KNN作为功能之一）
 - 混合搜索：在"关键词+向量"混合搜索场景表现优异
 - 适用场景：以文本搜索为主、向量搜索为辅的统一搜索平台需求
- 元数据管理方案



-
-
- 架构原则：FAISS专注向量检索，元数据交给专业数据库管理
- 存储方案：
 - 键值数据库（如Redis）：高性能简单元数据
 - 关系型数据库（如PostgreSQL）：复杂结构化元数据
 - 文档数据库（如MongoDB）：JSON格式元数据
- 更新机制：
 - 原始文本变更时才需更新向量
 - 仅元数据属性变更（如文件名、作者）不影响向量
- 关键技术实现



-
- IndexIDMap作用：实现向量与元数据ID映射的关键组件
- 多模态支持：
 - 可建立多个FAISS库（如文本库和图像库）
 - 图像处理推荐使用CLIP等跨模态Embedding模型
- 业务逻辑：
 - 向量用于相似度计算筛选片段
 - 元数据保存原始文本供大模型推理
 - 溯源时需要元数据标明信息来源
- 数据预处理：
 - 多种格式文件（PDF/Word/PPT）需先转为文本
 - 通过切分等处理后再存入向量数据库
- 案例分析与实践 02:52:18
 - 向量数据库与元数据管理实践



- 文本向量化: 使用text-embedding-v4模型将原始文档转化为向量表示
- 向量存储: 推荐使用Faiss进行向量存储和管理，因其轻量级且检索速度快
- 元数据管理: 可将元数据存储在metadata_store或Redis等数据库中
- 查询流程: 通过向量相似度匹配检索内容，并关联展示对应的元数据信息
- 三国演义文本Embedding实践
 - 数据处理: 使用word2vec工具对《三国演义》文本进行向量化处理
 - 模型选择: 基础实践可用简单模型，进阶学习可选用更大的预训练模型
 - 文件来源: 课程提供three_kingdoms源文件作为实践数据
- 向量数据库技术选型

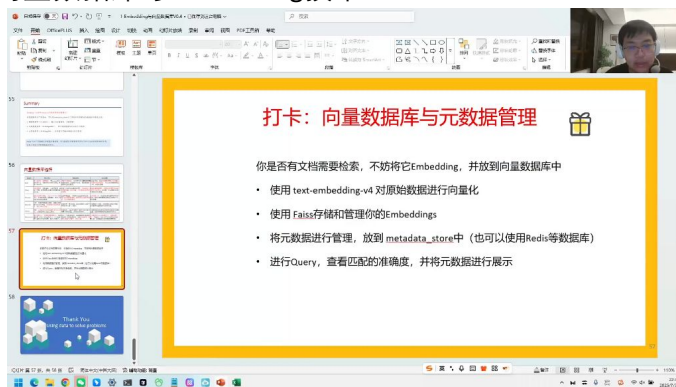


- FAISS:
 - 特点: Meta开发的算法库，支持CPU/GPU，提供前沿ANN算法
 - 适用场景: 需要深度集成向量检索能力的算法研究场景
- Milvus:
 - 特点: 开源云原生架构，支持多种ANN索引和参数调优
 - 适用场景: 大规模数据处理的企业级应用
- Pinecone:
 - 特点: 全托管服务，API简洁，低延迟检索
 - 适用场景: 快速验证AI应用的初创公司
- Weaviate:
 - 特点: 内置自动向量化模块，简化开发流程
 - 适用场景: 需要快速构建全链路应用的场景
- Qdrant:
 - 特点: Rust开发，内存安全，复杂过滤查询能力强
 - 适用场景: 对性能有极致要求的金融/电商场景
- Embedding技术问答
 - 与LLM的区别:
 - 功能差异: Embedding专注语义向量化，LLM侧重生成和理解
 - 成本差异: Embedding模型通常计算成本更低
 - 专业模型训练:
 - 流程: 使用word2vec或BERT在专业语料上训练
 - 部署: 可打包为可下载模型文件

- 维度说明:
 - 处理过程: 原始句子→分词→压缩到向量空间（如1024维）
- 推荐系统应用:
 - LLM作用: 用于高级语义理解和生成，与DeepFM等专有模型配合使用
- 元数据必要性:
 - 向量作用: 相似度计算和片段筛选
 - 元数据作用: 保存原始文本供LLM进一步处理
- 图像向量化工具
 - 工具推荐:
 - CLIP: 通用的图像-文本跨模态理解工具
 - DINOv2: Facebook开发的图像特征提取工具
 - 使用方式: 传入图片获取向量表示，后续处理流程与文本类似
 - 人脸特征:
 - 专用工具: 需使用专门的人脸特征提取库
 - 模型选择: 不同工具使用不同训练语料，如dash scope的text-embedding-v4专攻中文

8. 课程总结 02:56:19

1) 向量数据库与Embedding技术



- 核心概念：向量数据库是管理大量向量数据的专用数据库，作为大模型的"小助手"负责数据筛选工作
 - 数据处理流程：
 - 使用text - embedding - v4对原始文本进行向量化
 - 将生成的Embedding向量存入Faiss等向量数据库
 - 通过元数据管理(如Redis)存储关联信息
 - 应用价值：有效过滤无意义词汇（如英文中的"a/the/on/off"），提升大模型输入质量
- ### 2) 实践要点
- 训练选择：需要根据具体场景选择合适的Embedding模型进行训练
 - 元数据管理：建议将元数据存储专门的metadata_store中，便于后续查询验证
 - 效果验证：通过Query测试匹配准确度，并可视化展示元数据关联结果
- ### 3) 学习建议
- 练习重点：聚焦酒店案例中的两个典型场景进行实践
 - 学习路径：
 - 初期可先理解整体逻辑框架
 - 数学原理理解难度较大时可暂缓深入
 - 通过实际任务驱动逐步掌握核心技术

三、知识小结

知识点	核心内容	关键点	应用场景
-----	------	-----	------

Embedding概念	将高维数据压缩到低维空间的技术	- 解决稀疏性问题 - 26000维→1024维降维 - 语义特征保留	推荐系统、语义搜索、知识库构建
N元语法	文本特征提取方法	- 一元(单词)、二元(词组)、三元(组合) - TF-IDF计算特征权重 - 解决词序问题	酒店描述特征分析、商品标签生成
余弦相似度	向量相似度计算方法	- 夹角越小相似度越高(1→0→-1) - 公式： $\cos\theta=(A\cdot B)/(A B)$ - 语义相关性量化	句子相似度比较、推荐排序
Word2Vec	词向量训练工具	- 神经网络预测邻居词 - 三国演义案例训练 - 加减法语义运算(曹操+刘备-张飞)	文本特征提取、语义关系发现
向量数据库	高效相似检索系统	- Faiss(最快)、ES(混合搜索)、Milvus(分布式) - 十亿级数据毫秒响应 - 元数据管理(来源/作者)	知识库检索、跨模态搜索、实时推荐
RAG架构	检索增强生成技术	- 向量库筛选TOP5片段 - 作为上下文喂给LLM - 解决窗口限制问题	企业知识问答、精准报告生成
模型选型	Embedding模型选择	- 中文优选BGE - 多语言选paraphrase-multilingual - 套娃模型(齐娜AI)	跨境电商、多语言客服系统
性能优化	工程实践要点	- 768维性价比平衡点 - 测试集构建金标准 - CPU版Faiss轻量部署	社交媒体实时推荐、金融风控系统

以下为AI生成的图文笔记的内容

一、Embeddings和向量数据库08:43

1. Embedding 18:21

1) 嵌入的定义 18:22

>> 今天的学习目标

什么是Embedding

- 什么是Embedding
- N-Gram
- Word Embedding
- 余弦相似度计算
- Embedding模型的选择
- MTEB榜单
- 向量维度对模型性能的影响
- 神奇的“俄罗斯套娃”

向量数据库

- 什么是向量数据库
- FAISS, Elasticsearch, Milvus, Pinecone的特点
- 向量数据库与传统数据库的区别
- 如何将数据导入向量数据库
- Embedding与原数据导入Faiss
- 不同向量数据库的功能与性能比较



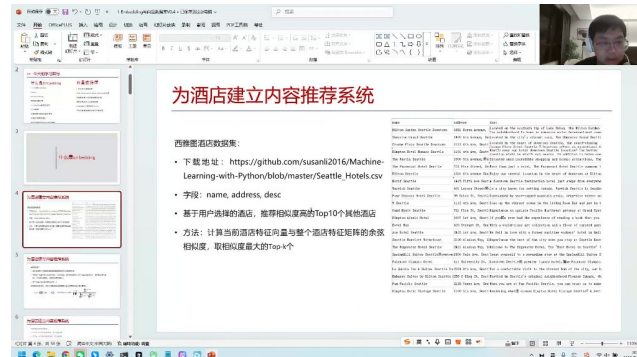
- 本质：将高维数据压缩到固定低维空间的过程
- 作用：通过体积压缩实现高效存储和计算
- 应用场景：信息存储管理、向量相似度计算（如余弦相似度）

2) Word Embedding 19:00

- 核心功能：将单词转化为向量表示
- 相似度计算：使用余弦相似度衡量向量间关系
- 模型选择维度：
 - 向量尺寸（1024/2048等）
 - 模型大小
 - 性能影响（不同维度对计算效率的影响）

3) 传统语义特征提取方法 20:33

- 学习目标与数据集介绍 20:36

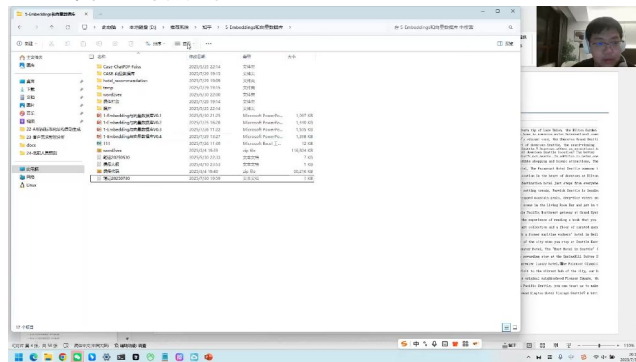


- 数据集：西雅图152家酒店数据（名称/地址/描述）
- 应用目标：基于文本相似度推荐Top10相似酒店
- 计算方法：特征向量与全量矩阵的余弦相似度计算
- 传统语义特征提取方法：分词 22:47
 - 基本单元：以单词为最小处理单位
 - 中文工具：结巴分词器
 - 示例：
 - 句子A："这个程序代码太乱那个代码规范" → ["这个","程序","代码","太乱","那个","代码","规范"]
 - 句子B："这个程序代码不规范那个更规范" → ["这个","程序","代码","不","规范","那个","更","规范"]
- 传统语义特征提取方法：特征值与向量 23:53
 - 特征构建：
 - 统计所有句子中的唯一单词作为特征维度
 - 计算每个句子在各维度上的出现频次
 - 向量表示：
 - 句子A向量：[1,1,2,1,1,1,0,0]（8维）
 - 句子B向量：[1,1,1,0,1,2,1,1]（8维）
- 传统语义特征提取方法：余弦相似度 25:46
 - 数学原理：
 - 计算公式：
$$\cos\theta = \frac{A \cdot B}{||A|| \times ||B||}$$
 - 取值范围：[-1,1]（1表示完全一致，-1表示完全相反）
 - 实际计算：
 - 示例句子计算结果：0.738
 - 解释：接近1表示语义高度相似
- 传统语义特征提取方法：存在的问题 30:35

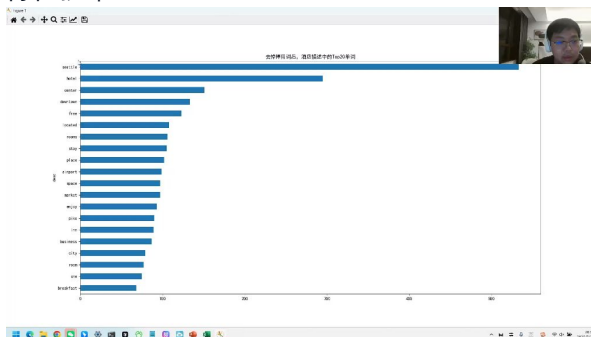
- **关键缺陷**：忽略词语顺序对语义的影响
- **反例证明**：
 - 调整语序后："这个程序代码规范那个更不规范"
 - 向量不变但实际语义相反
- **根本原因**：仅统计单词出现频次（一元语法）
- **传统语义特征提取方法**：N元语法 32:40
 - **改进方案**：引入词语组合特征
 - **具体方法**：
 - **二元语法**：统计相邻两个单词组合（如AB、BC）
 - **三元语法**：统计相邻三个单词组合（如ABC、BCD）
 - **优势**：能捕捉"不规范"与"规范不"等关键词序差异
 - **示例扩展**：
 - 序列"a b c d e"
 - 二元特征：ab, bc, cd, de
 - 三元特征：abc, bcd, cde

4) 应用案例 34:17

- **例题**：西雅图酒店数据集



- **数据准备**：使用西雅图酒店数据集，包含152家酒店的名称、地址和描述信息
- **数据特征**：主要分析酒店描述文本中的n-gram特征（一元、二元、三元语法）
- **文本特征提取**



- **n-gram分析**：
 - **一元语法(n=1)**：统计单个词频，"seattle"出现频率最高，其次是"hotel"、"center"等
 - **二元语法(n=2)**：分析词组组合，如"wifi"、"guest room"、"fitness center"等高频组合
 - **三元语法(n=3)**：提取三词短语，如"free wifi"、"seattle art museum"等特征短语
 - **n值选择**：通常取1-6，超过6后特征区分度降低
- **TF-IDF特征工程**

为酒店建立内容推荐系统



TF-IDF:

• TF: Term Frequency, 词频 $TF = \frac{\text{单词次数}}{\text{文档中总单词数}}$

一个单词的重要性和它在文档中出现的次数成正比。

• IDF: Inverse Document Frequency, 逆向文档频率

一个单词在文档中的区分度。这个单词出现的文档数越少, 区分

度越大, IDF越大 $IDF = \log \frac{\text{文档总数}}{\text{单词出现的文档数} + 1}$

■ ④ ⑤ ⑥ ⑦ ⑧ ⑨ ⑩ ⑪ ⑫ ⑬ ⑭ ⑮ ⑯ ⑰ ⑱ ⑲ ⑳ ㉑ ㉒ ㉓ ㉔ ㉕ ㉖ ㉗ ㉘ ㉙ ㉚ ㉛ ㉜ ㉝ ㉞ ㉟ ㊱ ㊲ ㊳ ㊴ ㊵ ㊶ ㊷ ㊸ ㊹ ㊺ ㊻ ㊼ ㊽ ㊾ ㊿

■ **TF(词频):** $TF = \frac{\text{单词出现次数}}{\text{文档总单词数}}$, 反映单词在文档中的重要性

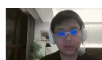
■ **IDF(逆向文档频率):** $IDF = \log(\frac{\text{文档总数}}{\text{单词出现的文档数} + 1})$, 衡量单词的区分度

■ **特征维度:** 一元+二元+三元语法共生成26,000+维特征向量

■ **特征矩阵:** 152家酒店×26,000维的稀疏矩阵

○ 相似度计算

为酒店建立内容推荐系统



```
def clean_text(text):
    # 全部小写
    text = text.lower()
    # ...
    return text

df['desc_clean'] = df['desc'].apply(clean_text)

# 使用TF-IDF提取文本特征
tf = TfidfVectorizer(analyzer='word', ngram_range=(1, 3), min_df=0, stop_words='en')
tfidf_matrix = tfidf.transform(df['desc_clean'])

print(tfidf_matrix)

print(tfidf_matrix.shape)

# 计算酒店之间的余弦相似度 (线性核函数)
cosine_similarities = linear_kernel(tfidf_matrix, tfidf_matrix)

print(cosine_similarities)
```

```
(152, 152) 0.02206477082716097
(152, 21971) 0.03622898061129
(152, 10060) 0.01709272121218795
(152, 22312) 0.01666017507678052
(152, 12462) 0.02588009709053069
(152, 12132) 0.026170114517605667
(152, 11079) 0.07109824862787955
(152, 11265) 0.0132272701890795
(152, 26870)
```

152家酒店, 之间的相似度矩阵
(1-Gram, 2-Gram, 3-Gram)

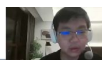
■ **余弦相似度:** 通过向量夹角计算酒店间的语义相似度

■ **相似度矩阵:** 152×152的对称矩阵, 对角线值为1 (自身相似度)

■ **推荐逻辑:** 对于目标酒店, 按相似度排序返回Top10最相似酒店

○ 推荐系统实现

为酒店建立内容推荐系统



```
# 基于相似度和酒店名称的推荐系统: 返回Top10酒店
def recommendations(name, cosine_similarities = cosine_similarities):
    recommended_hotels = []

    # 为酒店创建相似酒店列表并返回索引
    idx = indices(indices == name).index[0]
    print(idx)

    # 对152个酒店的相似分数进行降序排列从大到小进行排序
    score_series = pd.Series(cosine_similarities[idx].sort_values(ascending = False))
    top_10_indexes = list(score_series.index[1:11].index)

    # 获取相似酒店
    for i in top_10_indexes:
        recommended_hotels.append(df.index[i])

    return recommended_hotels

print(recommendations('Hilton Seattle Airport & Conference Center'))
```

```
idx: 0
["Embassy Suites by Hilton Seattle Tacoma International Airport",
 "DoubleTree by Hilton Hotel Seattle Airport", "Seattle Airport
 Marriott", "Marriott Seattle Sea-Tac Airport South", "Horne Lodge
 Seattle Airport North", "Four Points by Sheraton Downtown Seattle
 Center", "Knights Inn Seattle", "Horne Lodge Seattle-Bellevue",
 "Hampton Inn Seattle/Southcenter", "Radisson Hotel Seattle
 Airport"]
idx: 116
["15th Avenue Inn Bed and Breakfast", "Shafar Ballie Mansion Bed
 & Breakfast", "Chittenden House Bed and Breakfast", "Gastlight
 Inn", "Bed and Breakfast Inn Seattle", "Vikar (Land hotel),
 Seattle Broadway", "Hyatt House Seattle", "Mazart Guest House",
 "Quality Inn & Suites Seattle Center", "MarQueen Hotel"]
```

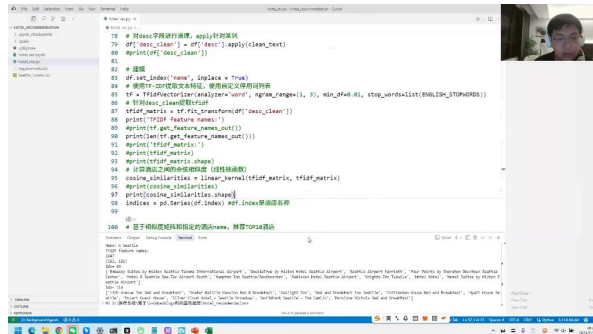
查找和指定酒店相似度最高的Top10家酒店

■ **实现步骤:**

- 数据清洗 (统一小写、去除停用词)
- 使用TF-IDF提取n-gram特征
- 计算余弦相似度矩阵
- 根据用户选择的酒店返回相似推荐

■ **效果验证:** 推荐结果与谷歌搜索结果吻合度较高, 验证了方法的有效性

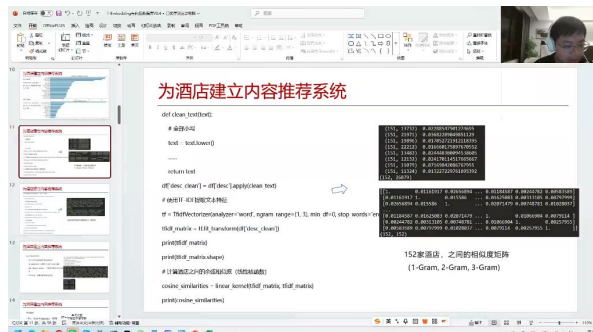
○ 代码关键点



- **CountVectorizer**: 用于n-gram特征提取
- **TfidfVectorizer**: 实现TF-IDF特征转换
- **linear_kernel**: 高效计算余弦相似度
- **pandas.Series**: 用于相似度排序和索引处理

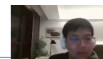
5) 为酒店建立内容推荐系统 47:23

- 基于内容的推荐
 - 文本特征提取



- **文本预处理**: 使用clean_text函数将文本全部转为小写，消除大小写差异
- **N-Gram特征**: 提取1-3元语法组合作为特征，共生成26000多维特征向量
- **TF-IDF计算**:
 - **TF(词频)**: $TF = \frac{\text{单词出现次数}}{\text{文档总单词数}}$ ，反映单词在文档中的重要性
 - **IDF(逆文档频率)**: $IDF = \log\left(\frac{\text{文档总数}}{\text{单词出现的文档数} + 1}\right)$ ，衡量单词区分度
 - 最终特征值为TF×IDF，未出现的单词特征值为0
- 相似度计算

为酒店建立内容推荐系统



TF-IDF:

• TF: Term Frequency, 词频 $TF = \frac{\text{单词次数}}{\text{文档中总单词数}}$

一个单词的重要性和它在文档中出现的次数呈正比。

• IDF: Inverse Document Frequency, 逆向文档频率

一个单词在文档中的区分度。这个单词出现的文档数越少，区分度越大，IDF越大

$$IDF = \log \frac{\text{文档总数}}{\text{单词出现的文档数} + 1}$$

- **稀疏性问题**: 26000维特征中大部分为0，矩阵非常稀疏
- **相似度度量**: 使用线性核函数计算余弦相似度，公式为
- **结果解释**: 相似度范围0-1，1表示完全相同，0表示完全不相关
- 推荐算法实现

为酒店建立内容推荐系统

基于内容的推荐系统

```
def recommendations(name, cosine_similarities = cosine_similarities):  
    recommended_hotels = []  
    # 找到指定酒店在相似矩阵中的索引  
    idx = indices(indices == name).index[0]  
    print(idx)  # 40  
    # 对152家酒店按相似矩阵中的相似程度从大到小进行排序  
    score_series = pd.Series(cosine_similarities[idx].sort_values(ascending=False))  
    # 取相似程度最大的10个 (跳过了自身)  
    top_10_indices = list(score_series.iloc[1:11].index)  
    # 按相似程度排序  
    for i in top_10_indices:  
        recommended_hotels.append(i)  # 40  
    return recommended_hotels  
print(recommendations("Hilton Seattle Airport & Conference Center"))
```

```
idx: 40  
["Embassy Suites by Hilton Seattle Tacoma International Airport",  
 "Qualtelway by Hilton Hotel Seattle Airport", "Seattle Airport  
 Marriott", "Metrol & Seattle Sea-Tac Airport South", "Como Lodge  
 Seattle Airport North", "Four Points by Sheraton Downtown Seattle  
 Center", "Knights Inn Tukwila", "Como Lodge Renton Bellevue",  
 "Hampton Inn Seattle/Southcenter", "Radisson Hotel Seattle  
 Airport"]  
idx: 116  
["15th Avenue Inn Bed and Breakfast", "Grafer Ballille Mansion Bed  
 & Breakfast", "Chittenden House Bed and Breakfast", "Gaslight  
 Inn", "Bed and Breakfast Inn Seattle", "Silver Strand Hotel  
 Seattle Broadway", "Hyatt House Seattle", "Hearst Guest House",  
 "Quality Inn & Suites Seattle Center", "Renaissance Hotel"]
```

查找和指定酒店相似度最高的Top10家酒店

推荐逻辑：

- 根据酒店名称找到对应索引
- 获取该酒店的相似度向量并降序排序
- 排除自身(相似度1.0)后取Top10

代码关键点：

- 使用pd.Series().sort_values(ascending=False)进行排序
- 通过iloc[1:11]跳过自身取前10个
- 示例输出显示机场酒店与B&B酒店各自形成聚类

算法优化思考

传统方法局限：

- 高维稀疏矩阵计算效率低
- 152家酒店即产生26000维，扩展性差

Embedding替代方案：

- 将特征降维为固定大小向量(如300维)
- 解决稀疏性问题，提升计算效率
- 任何对象均可向量化后进行相似度计算

核心流程总结

三步流程：

- 特征提取：N-Gram+TF-IDF生成特征矩阵
- 相似度计算：余弦相似度衡量酒店间相似性
- 推荐生成：按相似度排序输出Top-K结果

本质理解：

- 将酒店描述向量化
- 通过数学方法计算向量相似度
- 基于相似度进行推荐决策

6) Word Embedding总结 54:15

- 基于内容的推荐步骤

- Step1, 对酒店描述 (Desc) 进行特征提取
 - N-Gram, 提取N个连续字的集合, 作为特征
 - TF-IDF, 按照(min_df, max_df)提取关键词, 并生成TFIDF矩阵
- Step2, 计算酒店之间的相似度矩阵
 - 余弦相似度
- Step3, 对于指定的酒店, 选择相似度最大的Top-K个酒店进行输出

- 使用N-Gram方法提取N个连续字的集合作为特征
- 应用TF-IDF算法, 按照($min-df, max-df$)范围提取关键词并生成TF-IDF矩阵
- 相似度计算: 通过余弦相似度计算酒店之间的相似度矩阵
- 推荐输出: 对指定酒店选择相似度最大的Top-K个酒店进行推荐

- **降维策略**: 将高维稀疏特征（如N-Gram+TF-IDF）压缩到固定维度的向量（如1024维或128维）
- **计算优势**: 相比稀疏矩阵, Embedding向量之间更容易计算相似度
- **通用示例**: 以人物特征为例, 用向量表示性格（负值表示不好）和财富（正值表示有钱）

- **万物向量化**: 任何实体 (如人、单词) 都可表示为固定维度的向量
- **相似度计算**: 通过余弦相似度比较向量夹角, 如人物向量相似度计算示例
 - 蓝色向量(-0.4,0.8)与绿色向量(-0.3,0.2)相似度0.87
 - 蓝色向量与橙色向量(-0.5,-0.4)相似度-0.2

- **技术背景**：2014-2016年谷歌提出Word2Vec技术
- **训练方法**：将大不列颠百科全书输入神经网络进行无监督学习
- **向量表示**：每个单词学习出固定维度（如50维）的特征向量，数值范围在[-2,2]之间
- **可视化**：使用热力图展示向量各维度数值，红色表示高值，蓝色表示低值

- 我们将King与其他单词进行比较，可以看到Man和Woman更相近
- 同样有了向量，我们还可以进行运算

king-man+woman与queen的相似度最高

```
model.most_similar(positive=["king", "woman"], negative=["man"])
```

```
[('queen', 0.85236367749434),
 ('throne', 0.7664333581924438),
 ('prince', 0.7592144012455172),
 ('daughter', 0.747388302798747),
 ('alabaster', 0.7462218965076804),
 ('princess', 0.7424570322036743),
 ('kingdom', 0.7337411642074585),
 ('monarch', 0.7214490771292564),
 ('eldest', 0.72184829449468689),
 ('throne', 0.703396141008474)]
```

king - man + woman $\approx$ queen

- "Man"和"Woman"向量更相近
- 向量运算示例: $\text{king} - \text{man} + \text{woman} \approx \text{queen}$
- 相似度验证:
 - 计算结果与"queen"相似度最高 (0.852)
 - 其他相关词: "throne"(0.766),"prince"(0.759)等

Word Embedding

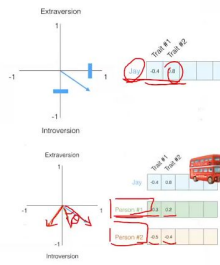
1020
128



什么是Embedding:

- 一种降维方式, 将不同特征转换为维度相同的向量
- 高维变量转换成one-hot => 维度非常高, 可以将其转换为固定size的embedding向量
- 任何物体, 都可以将它转换为向量的形式, 从Trait #1到 #N
- 向量之间, 可以使用相似度进行计算
- 当我们进行推荐的时候, 可以选择相似度最大的

cosine_similarity([0.4, 0.5], [0.5, 0.9]) = 0.87 ✓
cosine_similarity([0.4, 0.5], [-0.5, -0.4]) = -0.20



-
- **核心定义:** 将不同特征转换为相同维度的向量表示
- **转换优势:** 解决one-hot编码维度爆炸问题
- **计算特性:** 向量间可通过相似度计算建立关联关系
- 例题2: Word Embedding 59:13

Word Embedding

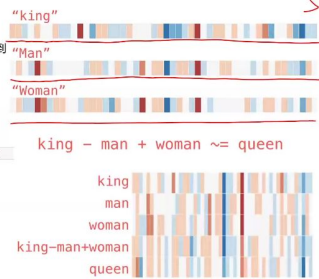


将Word进行Embedding:

- 我们将King与其他单词进行比较, 可以看到Man和Woman更相近
- 同样有了向量, 我们还可以进行运算

king-man+woman与queen的相似度最高

```
model.embed_kmlar(positions=["king", "man"], outputEmbed="man")
[{"queen": 0.85260347718034}, {"king": 0.799144012011272}, {"elizabeth": 0.74021980070088}, {"prince": 0.7084102201307}, {"morgan": 0.71711442140109}, {"michael": 0.72144977121949}, {"adam": 0.71880002948689}, {"victor": 0.709840501050741}]
```

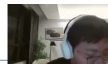


-
- **运算验证:** 再次演示king - man + woman $\approx$ queen的向量运算
- **相似度排序:** 列出与运算结果最相似的前10个单词及其相似度
- **Embedding的向量运算 59:25**
 - **数学基础:** 向量对位加减法保持维度不变 (如50维运算后仍为50维)
 - **物理意义:** 虽然计算机不理解语义, 但向量运算结果与人类认知一致
 - **典型示例:** king(国王) - man(男性) + woman(女性) $\approx$ queen(女王)的语义关系
- **Embedding的维度设置 01:01:26**
 - **可配置性:** 维度大小可自定义 (如50/100/200维)
 - **工具支持:** 现有工具支持用户自定义维度进行向量化
 - **维度特性:** 具体维度不代表特定语义, 仅是数学空间的压缩表示
- **Embedding的数学特性 01:02:00**
 - **抽象本质:** 对计算机而言仅是数字向量, 不关心具体语义
 - **应用原理:** 利用向量空间中的数学关系模拟语义关系
 - **神奇之处:** 纯数学运算能产生符合人类认知的结果, 这是AI强大表现的基础

7) Word2Vec 01:04:01

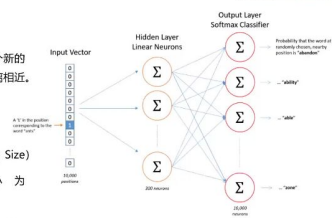
- **Word2Vec的定义**

Word Embedding



Word2Vec:

- 通过Embedding, 把原先词所在空间映射到一个新的空间中去, 使得语义上相似的单词在该空间内距离相近。
- Word Embedding => 学习隐藏层的权重矩阵
- 输入是one-hot编码
- 隐藏层的神经元数量为hidden_size (Embedding Size)
- 对于输入层和隐藏层之间的权值矩阵W, 大小为 [vocab_size, hidden_size]
- 输出层为[vocab_size]大小的向量, 每一个值代表着输出一个词的概率



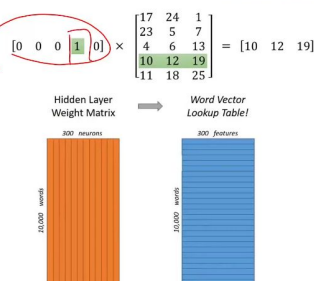
-

- **核心理念**：通过Embedding将词从原始空间映射到新空间，使语义相似的词在新空间中距离相近
- **输入输出**：
 - 输入：one-hot编码（如10000维向量，仅1个位置为1）
 - 输出：[vocab_size]大小的概率向量，表示预测词的概率分布
- **网络结构**：
 - 隐藏层神经元数=Embedding Size（如300）
 - 权重矩阵W大小：[vocab_size,hidden_size]
- Word Embedding到Word2Vec的过程 01:04:31

Word Embedding

对于输入的one-hot编码：

- 在矩阵相乘的时候，选取出矩阵中的某一行，而这一行就是输入词语的word2vec表示
- 隐含层的节点个数 = 词向量的维数
- 隐含层的输出是每个输入单词的Word Embedding
- word2vec，实际上就是一个查找表

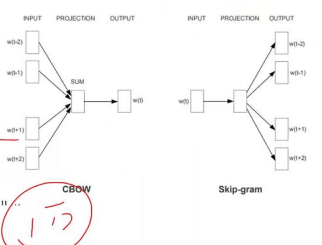


- 压缩原理：
 - 原始问题：one-hot编码过于稀疏（如10000维中仅1个1）
 - 解决方案：通过矩阵乘法将高维向量压缩到低维（如300维）
 - 数学实现：one-hot向量×权重矩阵W相当于查表操作
- 训练目标：
 - 自监督任务：预测上下文邻居词（如扣掉中心词让模型预测）
 - 优化目标：使预测概率分布与实际上下文分布一致
- 价值体现：
 - 存储效率：300维向量可保留99%信息量
 - 语义保持：压缩后相似词向量距离相近
- Word2Vec的两种模式 01:10:58

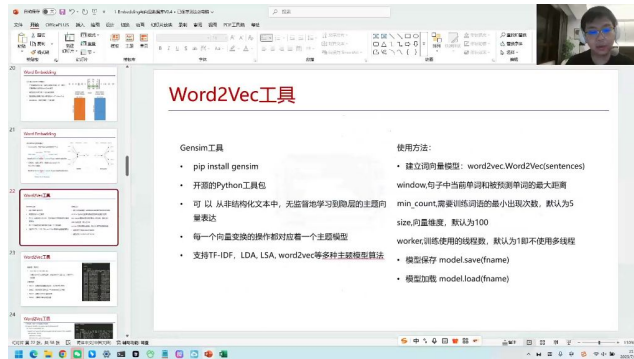
Word Embedding

Word2Vec的两种模式：

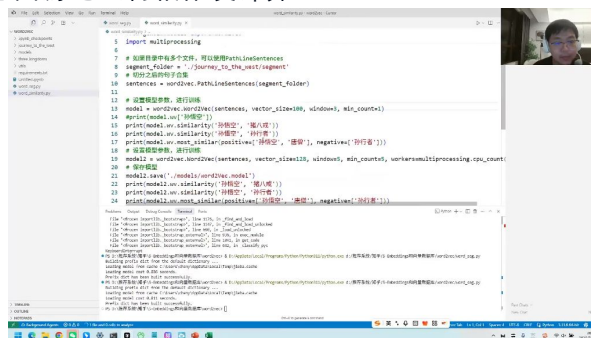
- Skip-Gram，给定input word预测上下文



- **Skip-Gram**：
 - 输入：中心词
 - 输出：预测上下文词的概率分布
 - 示例：给定"rests"，预测["he","in","life","peace"]的概率
- **CBOW**：
 - 输入：上下文词
 - 输出：预测中心词的概率
 - 示例：给定["he","in","life","peace"]，预测"rests"的概率
- 对比：两种模式输入输出相反，Skip-Gram更适合处理低频词
- Word2Vec工具 01:11:44

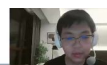


- 关键参数:
 - size: 向量维度 (默认100)
 - window: 上下文窗口大小
 - min_count: 词频阈值 (默认5)
 - workers: 并行线程数
- 操作接口:
 - 模型保存: model.save(filename)
 - 相似度计算: model.wv.similarity()
 - 向量运算: model.wv.most_similar(positive=[], negative=[])
- 例题:西游记人物相似度计算 01:11:54



- 实现步骤:
 - 使用jieba分词处理原文
 - 构建sentence迭代器
 - 训练Word2Vec模型 (设置size=100/128)
 - 计算词向量相似度
- 典型结果:
 - 孙悟空-猪八戒相似度: 0.939
 - 孙悟空-孙行者相似度: 0.983
 - 向量运算: 孙悟空+唐僧-孙行者≈"菩萨"(0.954)
- Word Embedding总结 01:19:17

Summary



Word2Vec工具的使用:

- Word Embedding就是将Word嵌入到一个数学空间里, Word2vec, 就是词嵌入的一种
- 可以将sentence中的word转换为固定大小的向量表达 (Vector Representations), 其中意义相近的词将被映射到向量空间中相近的位置。
- 将待解决的问题转换为单词word和文章doc的对应关系

大V推荐中, 大V => 单词, 将每一个用户关注大V的顺序 => 文章

商品推荐中, 商品 => 单词, 用户对商品的行为顺序 => 文章

- 核心价值:

- 将离散符号转为连续向量，保留语义关系
- 支持向量运算（如 $\text{king}-\text{man}+\text{woman}\approx\text{queen}$ ）
- 应用扩展：
 - 用户行为建模：用户关注序列 $\Rightarrow$ 文档
 - 商品推荐：用户行为序列 $\Rightarrow$ 文档
- 实现本质：
 - 通过神经网络学习压缩表示
 - 保持原始上下文预测任务的准确性

二、知识讲解01:23:30

1. 例题:三国演义嵌入 01:23:46

打卡：三国演义embedding

使用Gensim中的Word2Vec对三国演义进行Word Embedding

- 分析和曹操最相近的词有哪些
- 曹操+刘备-张飞=?

数据集：three_kingdoms.txt



- 实践目标：使用Gensim中的Word2Vec对《三国演义》进行词向量训练
- 分析任务：
 - 找出与"曹操"语义最相近的词语
 - 完成向量运算：曹操+刘备-张飞=?
- 数据集：three_kingdoms.txt（约27万字）
- 技术原理：通过词向量将文本特征压缩到低维空间（如100维），保留语义关系

2. Word2Vec工具的使用 01:24:45

1) Word2Vec工具简介 01:24:48

Word2Vec工具



```

# 将Word转换为Vec，然后计算相似度
from gensim.models import word2vec
import multiprocessing

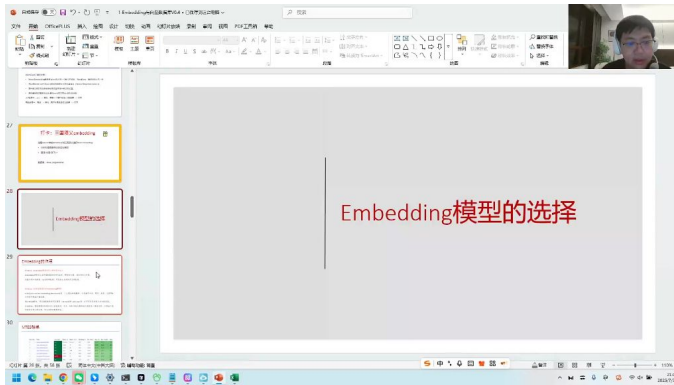
# 如果目录中有多个文件，可以使用PathLineSentences
sentences = word2vec.PathLineSentences('./segment')
# 设置模型参数，进行训练
model = word2vec.Word2Vec(sentences, size=100, window=3, min_count=1)
print(model.wv.similarity('孙悟空', '猪八戒'))
print(model.wv.similarity('孙悟空', '孙行者'))
# 设置模型参数，进行训练
model2 = word2vec.Word2Vec(sentences, size=128, window=5, min_count=5,
workers=multiprocessing.cpu_count())
# 保存模型
model2.save('./models/word2vec.model')
print(model2.wv.similarity('孙悟空', '猪八戒'))
print(model2.wv.similarity('孙悟空', '孙行者'))
print(model2.wv.most_similar(positive=['孙悟空', '唐僧'], negative=['孙行者']))
  
```

```

0.96224505
0.98238677
0.9193349
0.9293375
[('曹冲', 0.9546594613750977), ('何平', 0.9491345286369324),
 ('孟获', 0.9437925688997293), ('沙和尚', 0.94265623664855),
 ('唐僧', 0.9424765786623317), ('李靖', 0.9398664484812764),
 ('哪吒', 0.937991281877594), ('太子', 0.9359638809948486), ('众',
0.93202886203826), ('城隍', 0.9319333633896971)]
  
```

- 核心功能：
 - 将词语转换为固定维度的向量（如100维或128维）
 - 计算词语间的余弦相似度（0-1范围，越接近1越相似）
- 参数设置：
 - size：向量维度（典型值100/128/256）
 - window：上下文窗口大小（常用3-5）
 - min_count：词频阈值（过滤低频词）
- 典型应用：

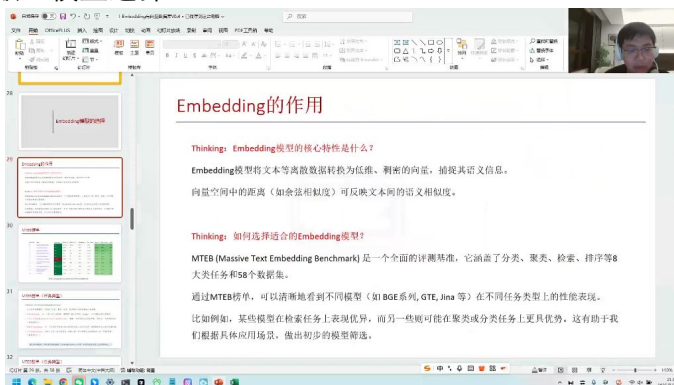
2) 例题2:Embedding模型压缩的规律 01:27:30



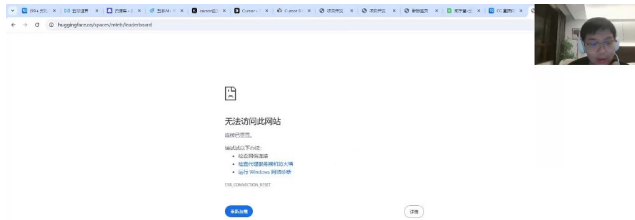
-
- **压缩原理：**
 - 将高维稀疏特征（如one-hot编码）转换为低维稠密向量
 - 典型压缩比例：1万维→300维（可根据需求调整）
- **维度选择：**
 - 常用2的幂次数：128/256/512/1024
 - 小规模语料可适当降低维度（如50-100维）
- **与大模型区别：**
 - **Embedding模型：** 特征提取器，输出固定维度向量
 - **大语言模型：** 生成式模型，处理复杂语义任务
- **模型选择依据：**
 - 参考MTEB评测基准（涵盖8类任务58个数据集）
 - 不同模型在不同任务表现各异（如BGE擅长检索，GTE长于分类）

3. Embedding的作用 01:27:48

1) 嵌入模型选择 01:28:36

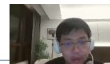


- - **本质特征：** Embedding模型是词压缩模型，通过将离散数据转换为低维稠密向量来捕捉语义特征，而非生成式模型
 - **训练方法：** 使用watch wack工具训练，通过大量语料迭代学习矩阵参数（初始随机→逐步优化）
 - **典型应用：** 适用于分类、检索、排序等特征抽取任务，不能直接生成内容
 - **性能差异：** 不同模型在语义理解精度上有显著差异，需根据下游任务选择
- #### 2) MTEB榜单 01:29:46
- MTEB介绍 01:29:49



-
- **基准构成**：涵盖8大类任务和58个数据集，包括分类/聚类/检索/排序等
- **模型对比**：可横向比较BGE/GTE/Jina等系列模型在不同任务的表现
- **使用方式**：通过huggingface平台访问MTEB榜单（需特殊网络环境）
- **MTEB榜单（任务类型）** 01:32:04

MTEB榜单（任务类型）



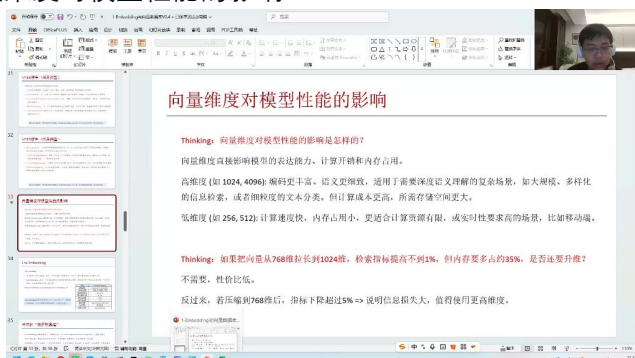
MTEB (Massive Text Embedding Benchmark) :

一个全面的评测基准，它涵盖了分类、聚类、检索、排序等8大类任务和58个数据集。

- **检索 (Retrieval)**：从一个庞大的文档库中，根据用户输入的查询 (Query)，找出最相关的文档列表。
- **语义文本相似度 (Semantic Textual Similarity, STS)**：判断一对句子的语义相似程度，并给出一个连续的分（例如1到5分）。
- **重排序 (Reranking)**：对一个已经初步检索出的文档列表进行二次优化排序，使得最相关的文档排在最前面。
- **分类 (Classification)**：将单个文本（如电影评论、新闻文章）划分到预定义类别中（如“正面/负面”、“体育/科技”）。

通过MTEB榜单，可以看到不同模型（如BGE系列、GTE、Jina等）在不同任务类型上的性能表现。

-
- **检索任务(Retrieval)**：从文档库中找出与查询最相关的文档列表
- **语义相似度(STS)**：评估句子对的语义相似程度（1-5分连续评分）
- **重排序(Reranking)**：对初步检索结果进行优化排序
- **分类任务**：将文本划分到预定义类别（如情感分析）
- **聚类任务**：无监督地将相似文本自动分组
- **双语挖掘**：跨语言语义匹配
- **向量维度对模型性能的影响** 01:35:58



-
- **高维度优势**：1024/4096维能编码更丰富语义，适合复杂场景（如细粒度分类）
- **低维度优势**：256/512维计算速度快，适合实时性要求高的场景（如移动端）
- **选择标准**：维度提升1%性能但增加35%内存时不建议升维；降维导致>5%性能损失时应保持高维
- **典型案例**：Qwen3模型有不同维度版本（0.6B/4B/8B），4096维版本效果最优但成本最高
- **相关问题解答** 01:36:18

Extension this Benchmark

Advanced Model Filters

Summary Performance per Task Task Information

Rank (No.)	Model	Zero-shot	Clustering	Instruction S.	Multilabel class.	Pair Classification	Reasoning	Retrieval
1	gpt4o-embedding-001	99%	94.34	5.74	99.54	99.63	94.34	97.71
2	gpt4o-embedding-002	99%	97.44	14.04	99.44	99.54	94.34	97.44
3	gpt4o-embedding-003	99%	97.74	14.04	99.44	99.54	94.34	97.44
4	gpt4o-embedding-004	99%	97.74	14.04	99.44	99.54	94.34	97.44
5	gpt4o-embedding-005	99%	97.74	14.04	99.44	99.54	94.34	97.44
6	gpt4o-embedding-006	99%	97.74	14.04	99.44	99.54	94.34	97.44
7	gpt4o-embedding-007	99%	97.74	14.04	99.44	99.54	94.34	97.44
8	gpt4o-embedding-008	99%	97.74	14.04	99.44	99.54	94.34	97.44
9	gpt4o-embedding-009	99%	97.74	14.04	99.44	99.54	94.34	97.44
10	gpt4o-embedding-010	99%	97.74	14.04	99.44	99.54	94.34	97.44
11	gpt4o-embedding-011	99%	97.74	14.04	99.44	99.54	94.34	97.44
12	gpt4o-embedding-012	99%	97.74	14.04	99.44	99.54	94.34	97.44
13	gpt4o-embedding-013	99%	97.74	14.04	99.44	99.54	94.34	97.44
14	gpt4o-embedding-014	99%	97.74	14.04	99.44	99.54	94.34	97.44
15	gpt4o-embedding-015	99%	97.74	14.04	99.44	99.54	94.34	97.44
16	gpt4o-embedding-016	99%	97.74	14.04	99.44	99.54	94.34	97.44
17	gpt4o-embedding-017	99%	97.74	14.04	99.44	99.54	94.34	97.44
18	gpt4o-embedding-018	99%	97.74	14.04	99.44	99.54	94.34	97.44
19	gpt4o-embedding-019	99%	97.74	14.04	99.44	99.54	94.34	97.44
20	gpt4o-embedding-020	99%	97.74	14.04	99.44	99.54	94.34	97.44

Download Table

- 矩阵学习：Embedding矩阵通过数百次迭代训练获得，非人工设定
- 分词模型关系：与token分词模型无绑定关系，两者训练目标和参数不共享
- 视频处理：视频Embedding常用CLIP或DINO模型
- 逻辑体现：50维即可体现基础逻辑，更高维度（如1000维）能提升精细度

● Embedding和LLM模型的区别 01:39:08

Extension this Benchmark

Advanced Model Filters

Summary Performance per Task Task Information

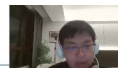
Rank (No.)	Model	Zero-shot	Clustering	Instruction S.	Multilabel class.	Pair Classification	Reasoning	Retrieval
1	gpt4o-embedding-001	99%	94.34	5.74	99.54	99.63	94.34	97.71
2	gpt4o-embedding-002	99%	97.44	14.04	99.44	99.54	94.34	97.44
3	gpt4o-embedding-003	99%	97.74	14.04	99.44	99.54	94.34	97.44
4	gpt4o-embedding-004	99%	97.74	14.04	99.44	99.54	94.34	97.44
5	gpt4o-embedding-005	99%	97.74	14.04	99.44	99.54	94.34	97.44
6	gpt4o-embedding-006	99%	97.74	14.04	99.44	99.54	94.34	97.44
7	gpt4o-embedding-007	99%	97.74	14.04	99.44	99.54	94.34	97.44
8	gpt4o-embedding-008	99%	97.74	14.04	99.44	99.54	94.34	97.44
9	gpt4o-embedding-009	99%	97.74	14.04	99.44	99.54	94.34	97.44
10	gpt4o-embedding-010	99%	97.74	14.04	99.44	99.54	94.34	97.44
11	gpt4o-embedding-011	99%	97.74	14.04	99.44	99.54	94.34	97.44
12	gpt4o-embedding-012	99%	97.74	14.04	99.44	99.54	94.34	97.44
13	gpt4o-embedding-013	99%	97.74	14.04	99.44	99.54	94.34	97.44
14	gpt4o-embedding-014	99%	97.74	14.04	99.44	99.54	94.34	97.44
15	gpt4o-embedding-015	99%	97.74	14.04	99.44	99.54	94.34	97.44
16	gpt4o-embedding-016	99%	97.74	14.04	99.44	99.54	94.34	97.44
17	gpt4o-embedding-017	99%	97.74	14.04	99.44	99.54	94.34	97.44
18	gpt4o-embedding-018	99%	97.74	14.04	99.44	99.54	94.34	97.44
19	gpt4o-embedding-019	99%	97.74	14.04	99.44	99.54	94.34	97.44
20	gpt4o-embedding-020	99%	97.74	14.04	99.44	99.54	94.34	97.44

Download Table

- 核心差异：
 - 任务类型：Embedding是语义理解型AI，LLM是生成型AI
 - 成本对比：deepseek-r1 671B模型成本比4B/8B模型高100倍
 - 响应速度：Embedding处理约100ms，LLM同任务需50s以上
- 应用场景：
 - Embedding适合知识库检索等特征抽取任务
 - LLM适合需要内容生成的复杂任务
- 性能平衡：LLM理解精度可能高5-10%，但时间成本呈指数级增长

3) 向量维度对模型性能的影响 01:41:32

向量维度对模型性能的影响



Thinking：向量维度对模型性能的影响是怎样的？

向量维度直接影响模型的表达能力、计算开销和内存占用。

高维度 (如 1024, 4096): 编码更丰富、语义更细致，适用于需要深度语义理解的复杂场景，如大规模、多样化的信息检索，或者细粒度的文本分类。但计算成本更高，所需存储空间更大。

低维度 (如 256, 512): 计算速度快，内存占用小，更适合计算资源有限，或实时性要求高的场景，比如移动端。

Thinking：如果把向量从768维拉长到1024维，检索指标提高不到1%，但内存要多占约35%，是否还要升维？

- 维度与性能关系：向量维度直接影响模型的表达能力、计算开销和内存占用
- 高维度特点：
 - 典型维度：1024/4096维
 - 优势：编码更丰富、语义更细致
 - 适用场景：大规模信息检索、细粒度文本分类等复杂任务
 - 代价：计算成本高，存储空间需求大 (如768维→1024维内存增加35%)

- **低维度特点：**
 - 典型维度：256/512维
 - 优势：计算速度快，内存占用小
 - 适用场景：移动端应用、实时性要求高的系统
- **性价比考量：**升维决策需权衡指标提升幅度（如检索指标提高<1%）与资源消耗增加比例

4) 神奇的俄罗斯套娃 01:43:48

- **俄罗斯套娃模型：Matryoshka Representation Learning (MRL)**

神奇的“俄罗斯套娃”



Jina-embeddings训练时使用了一种特殊技术（Matryoshka Representation Learning, MRL）=> 俄罗斯套娃

生成完整向量：模型总是先在内部生成一个最完整、维度最高（比如 2048维）的向量。

按需截断：这个长向量有一个非常神奇的特性，它的前128维、前256维、前512维.....本身就是一组高质量的、独立的、可以正常使用的短向量。

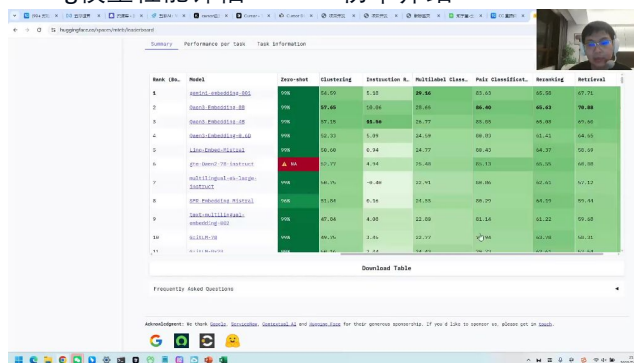


用户指定：当调用模型时，可以通过 `embedding_size` 告诉模型“我这次只需要前512维就够了”=> 模型就会截断向量，只返回前512维

- 
- **核心机制：**
 - 先生成完整高维向量（如2048维）
 - 支持按需截取前N维（128/256/512维等）
- **技术优势：**
 - 各维度段均可独立使用
 - 截断后质量损失小（前128维仍保持高质量）
- **实现方式：**通过`embedding_size`参数指定所需维度
- **套娃模型的应用场景 01:44:54**
 - **社交媒体场景：**
 - 推荐使用：128维向量
 - 优势：实现实时评论检索（用户问题与评论均用128维）
 - **金融分析场景：**
 - 案例：私募基金投研报告分析
 - 推荐使用：高精度维度（日耗约7000美金）
 - 价值：精准匹配百万份文档，支撑投资决策
- **Embedding模型作为大模型的辅助工具 01:46:35**
 - **角色定位：**
 - 功能：语义理解与信息过滤
 - 优势：突破大模型上下文窗口限制
 - **工作流程：**
 - 向量化处理语料
 - 相似度计算筛选
 - 将精华片段输入大模型
 - **典型差异：**
 - 大模型：生成内容（如写报告）
 - Embedding模型：理解内容（相似度计算）
- **模型尺寸的选择：任务价值与需求 01:47:55**
 - **决策要素：**
 - 任务价值：高价值任务倾向高维（如投研报告可接受周级产出）
 - 实时性要求：实时系统选择低维
 - **典型案例：**

- 快速响应：选择128维（Jina模型截断）
 - 深度分析：使用完整维度（如2000维）
- 单语言与多语言模型的选择 01:48:31
 - 单语言模型：
 - 典型代表：bge-large-ch（中文专用）
 - 优势：本土化表达精准（如"七天无理由退货"）
 - 适用场景：单一语言服务（如中文客服系统）
 - 多语言模型：
 - 典型场景：跨国连锁酒店评论分析
 - 优势：跨语言语义对齐（clean room ≈ 干净房间）
 - 缺点：单语种精度稍逊
- 多语言模型：应用场景与优缺点 01:49:35
 - 实现原理：
 - 多语言联合训练
 - 共享向量空间映射
 - 典型应用：
 - 跨语言检索（中文查询匹配日文评论）
 - 多语种知识库构建

● Embedding模型性能评估：MTEB榜单介绍 01:51:31



Rank (th.)	Model	Zero-shot	Clustering	Instruction R.	Multilingual class.	Pairs Classification	Summarization	Retrieval
1	OpenAI GPT-4o	99%	99.9%	99.9%	99.9%	99.9%	99.9%	99.9%
2	OpenAI GPT-4o-mini	99%	99.9%	99.9%	99.9%	99.9%	99.9%	99.9%
3	OpenAI GPT-4o	99%	99.9%	99.9%	99.9%	99.9%	99.9%	99.9%
4	OpenAI GPT-4o	99%	99.9%	99.9%	99.9%	99.9%	99.9%	99.9%
5	OpenAI GPT-4o	99%	99.9%	99.9%	99.9%	99.9%	99.9%	99.9%
6	OpenAI GPT-4o	99%	99.9%	99.9%	99.9%	99.9%	99.9%	99.9%
7	OpenAI GPT-4o	99%	99.9%	99.9%	99.9%	99.9%	99.9%	99.9%
8	OpenAI GPT-4o	99%	99.9%	99.9%	99.9%	99.9%	99.9%	99.9%
9	OpenAI GPT-4o	99%	99.9%	99.9%	99.9%	99.9%	99.9%	99.9%
10	OpenAI GPT-4o	99%	99.9%	99.9%	99.9%	99.9%	99.9%	99.9%

- 榜单特点：
 - 覆盖100+模型、1000+语言
 - 包含131个任务、9种任务类型
- 使用注意：
 - 模型表现存在任务差异（如检索vs分类）
 - 仅作参考（测试集可能与业务场景不符）
- 模型选型的关键步骤 01:52:29

如何选择适合的Embedding模型

Thinking：如何选择适合的Embedding模型，依靠MTEB就可以？

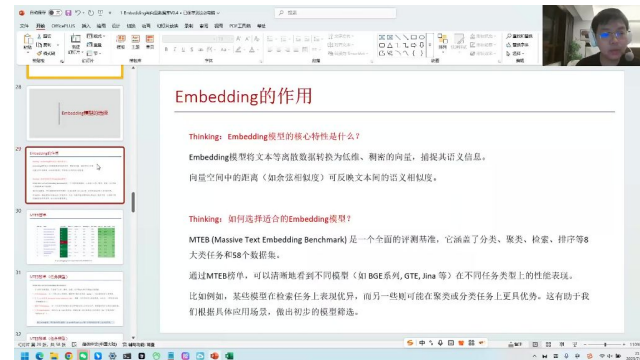
模型选型是一个系统的过程，不能仅仅依赖于公开榜单。包括以下关键步骤：

- 明确业务场景与评估指标：首先定义核心任务是检索、分类还是聚类？并确定衡量业务成功的关键指标，如搜索召回率（Recall@K）、准确率（Accuracy）或 NDCG。
- 构建“黄金”测试集：准备一套能真实反映您业务场景和数据分布的高质量小规模测试集。比如，构建一系列“问题-标准答案”对 => 评估模型好坏的“金标准”。
- 小范围对比测试（Benchmark）：从MTEB榜单中挑选几款排名靠前且符合需求（如语言、维度）的候选模型。使用“黄金”测试集，对这些模型进行评测。

Embedding模型的选择属于综合评估，即结合测试结果、模型的推理速度、部署成本 => 做出最终决策

- 四步法：
 - 明确业务指标（Recall@K/Accuracy/NDCG）
 - 构建黄金测试集（100+高质量样本）
 - 小范围对比测试（结合MTEB初筛）

- 综合评估（质量+速度+成本）
- 核心原则：
 - 拒绝唯榜单论
 - 建立业务专属评估标准
- Embedding模型的作用与核心特性 01:55:28



- 核心功能：
 - 离散数据→稠密向量（维度压缩）
 - 语义相似度计算（余弦相似度）
- 评估体系：
 - MTEB基准：8类任务/58个数据集
 - 典型任务差异：检索vs聚类vs分类

4. 休息 01:55:56

1) 休息五分钟 01:56:06

2) 休息结束，准备继续 02:00:52

5. 答疑 02:01:08

- **Embedding模型套娃**：可以嵌套使用多个独立的embedding模型，512维度的向量也是可行的特殊技术方案
- **向量与原文转换**：通过向量数据库同时存储原文和向量化表示，实现双向转换管理

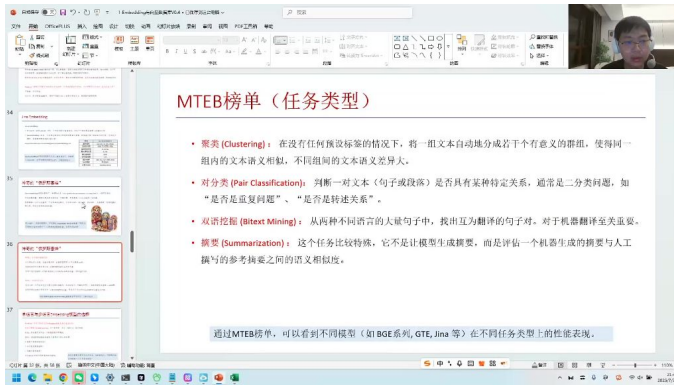
6. Embedding与向量数据库

1) Embedding核心作用

Rank (Overall)	Model	Zero-shot	Meaning S.	Relation S.	Cluster S.	Ranking S.	Top Tokens	Mean ST	Mean T2B2	STB
1	openai.embedding-ada-002	99%	Unknown	Unknown	99%	99%	2560	99.97	99.99	99.9
2	openai.embedding-ada-001	99%	99%	99%	99%	99%	2560	99.94	99.94	99.9
3	openai.embedding-ada-003	99%	99%	99%	99%	99%	2560	99.93	99.94	99.9
4	openai.embedding-ada-004	99%	99%	99%	99%	99%	2560	99.93	99.94	99.9
5	openai.embedding-ada-005	99%	99%	99%	99%	99%	2560	99.93	99.94	99.9
6	openai.embedding-ada-006	99%	99%	99%	99%	99%	2560	99.93	99.94	99.9
7	openai.embedding-ada-007	99%	99%	99%	99%	99%	2560	99.93	99.94	99.9
8	openai.embedding-ada-008	99%	99%	99%	99%	99%	2560	99.93	99.94	99.9
9	openai.embedding-ada-009	99%	99%	99%	99%	99%	2560	99.93	99.94	99.9
10	openai.embedding-ada-010	99%	99%	99%	99%	99%	2560	99.93	99.94	99.9

- **语义关系判断**：通过向量化判断上下文语义的关联程度
- **模型性能对比**：不同模型在MTEB榜单上的表现差异显著，如Qwen3-Embedding-4B在多个任务中表现突出

2) MTEB任务类型



- - **聚类任务:** 无监督地将语义相似的文本自动分组
 - **对分类任务:** 判断文本对是否具有特定关系（如重复问题识别）
 - **双语挖掘:** 跨语言寻找互为翻译的句子对
 - **摘要评估:** 量化机器生成摘要与人工参考摘要的语义相似度
- 3) 向量数据库核心概念

向量数据库



Thinking: 如何让LLM“记住”并利用海量、多样的私有知识？

向量数据库: AI时代的核心记忆体

与传统的关系型数据库不同, 向量数据库用于存储和查询由非结构化数据 (如文本、图片、音视频) 转化而来的高维向量嵌入 (Embeddings)。这些向量在多维空间中的距离代表了原始数据的语义相似度。

因此, 向量数据库的核心能力是高效的相似性检索。

Thinking: 向量数据库的核心价值?

- 为大模型提供长期记忆: 弥补LLM上下文窗口 (Context Window) 长度限制和知识更新延迟的问题。
- 实现私有知识库的问答与搜索: 将企业内部文档、产品信息等转化为向量, 实现基于语义的智能检索。
- 赋能推荐系统、以图搜图等多种应用: 通过计算用户、物品的向量相似度, 提供更精准的推荐。

-
- **与传统数据库区别:**
 - 存储内容: 高维向量而非结构化数据
 - 查询方式: 近似匹配 (余弦相似度) 而非精确匹配
 - 返回结果: 语义相关的多个结果而非单条记录
- **核心价值:**
 - 扩展LLM记忆: 突破上下文窗口限制
 - 实现语义搜索: 支持私有知识库智能检索
 - 赋能推荐系统: 基于向量相似度的物品推荐

4) 主流向量数据库对比

- **Faiss:**
 - 特点: 轻量级专注相似度检索
 - 优势: 查询速度极快
- **Elasticsearch:**
 - 特点: 支持混合检索 (关键词+语义)
 - 优势: 商业场景成熟度高
- **Milvus:**
 - 特点: 基于Go语言的分布式架构
 - 优势: 海量数据分布式处理
- **Pinecone:**
 - 特点: 云原生托管服务
 - 优势: 部署维护简便

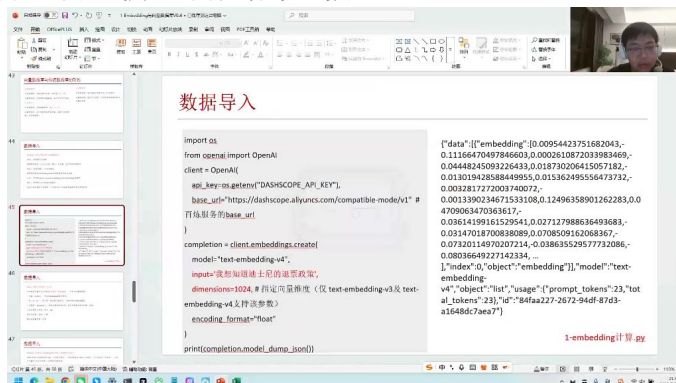
5) 向量化技术应用

- **多模态支持:**
 - 文本向量化: 如BERT、GPT等模型

- 图像向量化：如ResNet、CLIP等模型
- 特征提取原理：
 - 通过预训练模型获取抽象特征表示
 - 不同领域需使用专用模型（如ResNet用于图像分类）

7. 向量数据库 02:02:26

1) 例题:向量模型与数据库连接 02:11:47



- Embedding模型核心功能
 - 语义向量化：无论输入文本长度（从单个词"迪士尼"到2000字文章），均可通过embedding模型转化为固定维度（如1024维）的向量表示
 - 维度压缩特性：长文本压缩为低维向量时会产生语义损失，但保留了核心语义特征
 - 模型兼容性：阿里云text-embedding-v4模型支持自定义维度参数（v3/v4版本特有功能）

● 实践示例

```
1 import os
2 from openai import OpenAI
3 client = OpenAI(api_key=os.getenv("DASHSCOPE_API_KEY"),
4                 base_url="https://dashscope.aliyuncs.com/compatible-mode/v1")
5
6 completion = client.embeddings.create(
7     model='text-embedding-v4',
8     input='我想知道迪士尼的退票政策',
9     dimensions=1024, # 指定向量维度（仅text-embedding-v3及text-embedding-v4支持该参数）
10    encoding_format="float"
11 )
12 print(completion.model_dump_json())
```

- 参数说明：
 - dimensions=1024：指定生成1024维浮点型向量
 - encoding_format="float"：确保输出为浮点数格式
- 执行效率：国内网络调用阿里云API时响应速度较优
- 输出特征：生成向量包含正负浮点数（如0.00954, -0.11166等），反映多维语义特征
- 扩展问答

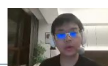
- 模型训练：专业领域数据需通过特定语料训练才能生成适配的embedding模型
- 部署方案：训练完成的模型可通过标准化接口封装供外部调用

2) 有人专业性资料，如何从零开始训练出自己的embedding模型 02:14:48

- 训练工具与模型选择 02:14:53
 - Word2vec工具：可用于简单训练任务，但精度相对较低，适合基础场景使用
 - BERT模型应用：2018年流行的神经网络模型（非大语言模型），可通过语料训练生成特定模型
 - 模型选择原则：根据任务复杂度选择，简单任务可用Word2vec，复杂任务建议使用BERT等神经网络

- **神经网络压缩**：训练后的模型可进行压缩处理，便于后续任务部署和执行
- **分词与向量压缩 02:15:45**
 - **分词过程**：原始句子→分词→得到空格分隔的词语序列（如abcd）
 - **向量压缩**：将分词结果映射到隐空间（如1024维向量），实现对整体语义的理解
 - **分词规则**：中文分词依赖词典（如结巴分词），可自定义添加专有名词词典
 - **Transformer优势**：相比传统方法能更好地处理语义理解任务
- **神经网络模型与推荐系统 02:17:01**
 - **推荐系统架构**：
 - 传统方案：双塔神经网络结构、DeepFM、NFM、Wide&Deep等专用模型
 - 大模型方案：可作为辅助工具，非必需组件
 - **模型特点**：专用模型针对特定数据训练，预测用户偏好，非通用大模型
 - **发展历程**：推荐系统在大模型兴起前已广泛应用神经网络技术
- **神经网络模型与大模型辅助 02:18:13**
 - **大模型辅助功能**：
 - 特征标记：为物品打语义标签
 - 特征抽取：提取物品关键特征
 - 话术生成：解释推荐理由
 - **应用定位**：主要承担特征处理工作，不直接用于推理
 - **工业案例**：制造企业用元数据管理产品型号（如触摸屏型号A/B区分）
- **FAISS库的安装与使用 02:19:03**
 - **安装方式**：pip install faiss-cpu（CPU版）或pip install faiss-gpu（GPU版）
 - **核心功能**：高效向量相似度计算库，支持十亿级文档毫秒级检索
 - **版本选择**：根据硬件配置选择对应版本，无GPU设备安装CPU版本即可
- **元数据的定义与管理 02:19:45**
 - **元数据定义**：描述**向量的附加信息**，包括：
 - 数据来源（文件名/网页/章节）
 - 业务属性（商品类别/型号/价格）
 - 管理信息（创建日期/作者）
 - **工业案例**：制造企业将产品型号作为关键元数据（如触摸屏型号A/B）
 - **存储必要性**：向量化后丢失原始信息，需通过元数据保留关键业务属性
- **元数据管理实现 02:22:48**

数据导入



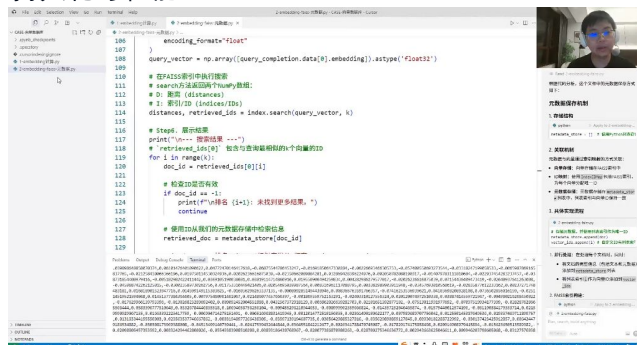
Step3：数据与元数据一同导入

将生成的向量及其关联的**元数据（Metadata）**一同存入向量数据库。

- 向量（Vector）：生成的Embedding数字数组。
- 唯一ID（ID）：用于唯一标识每个数据点，方便后续的更新或删除。
- 元数据（Metadata）：描述向量的附加信息，是实现高级检索的关键。例如：
 - 文本来源的文件名、章节、URL
 - 商品的类别、品牌、价格
 - 图片的创建日期、作者

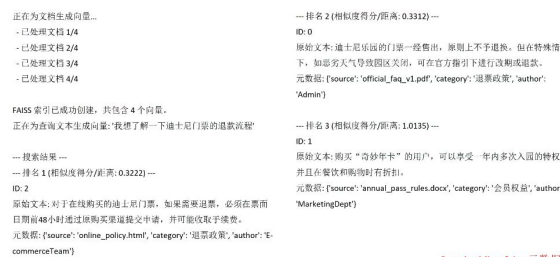
-
- **三要素存储**：
 - 向量：Embedding数字数组
 - 唯一ID：数据点标识符
 - 元数据：实现高级检索的关键
- **实现方式**：使用FAISS的**IndexIDMap**建立向量与元数据的映射关系
- **类比说明**：类似数据库管理，新增数据自动生成唯一ID
- **代码实现与向量索引 02:24:08**
 - **技术栈**：

- 使用DashScope的text-embedding-v4模型（兼容OpenAI接口）
- 生成1024维向量
- 数据处理流程：
 - 原始文档准备（含元数据标注）
 - 调用Embedding接口生成向量
 - 构建向量列表和元数据存储
- 索引优化：采用精确索引方案，平衡检索精度与速度
- 相似向量检索与结果展示 02:29:20
 - 检索流程：
 - 用户问题向量化（需使用相同模型）
 - FAISS的search函数执行K近邻搜索（如K=3）
 - 通过ID关联获取元数据和原始文本
 - 关键参数：
 - 维度一致性：必须保持1024维
 - 模型一致性：需使用相同embedding模型
 - 结果展示：输出相似度分数及对应元数据，支持业务决策
- FAISS持久化与性能 02:31:10



- 持久化能力：FAISS索引可以保存为.pkl文件，下次直接读取使用，加载速度非常快
- 运行机制：实际运行时索引数据加载到内存中操作，但支持文件持久化存储
- 性能表现：官方支持十亿级(e级别)向量检索，典型响应时间在几百毫秒量级
- 检索结果解析

Embedding与原数据导入Faiss



- 距离含义：返回的相似度得分实际是距离值，距离越大表示相似度越低
- 结果排序：示例中k=3返回前三个结果，距离分别为0.3222、0.3312、1.0135
- 元数据结构：包含原始文本和metadata字典（source/category/author等字段）
- 典型应用流程
 - 数据处理阶段：
 - 并行处理：同时将文档信息存入metadata_store列表，并将列表索引作为向量ID

- **索引构建**：使用IndexFlatL2创建精确索引，通过IndexIDMap封装实现ID映射
- **查询阶段**：
 - **向量转换**：将用户问题转换为相同维度的向量
 - **快速检索**：FAISS返回相似向量ID和距离值
 - **结果关联**：通过ID从metadata_store获取完整信息
- 元数据管理的更健壮方案 02:36:16

Summary



Thinking: 如何将metadata元数据管理的更健壮？

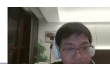
在更复杂的生产环境中，可以将 metadata_store 这个简单的列表替换为更健壮的存储方案：

- 键值数据库（如 Redis）：通过ID快速查询，性能极高。
- 关系型数据库（如 PostgreSQL）：可以存储更复杂的结构化元数据。
- 文档数据库（如 MongoDB）：非常适合存储JSON格式的元数据。

FAISS 专注于其最擅长的高速向量检索，而元数据的存储和管理可以交给专业的数据库系统负责，实现了架构上的解耦和高效协同。

-
- **基础方案局限**：使用Python列表(metadata_store)临时存储元数据，通过列表索引关联
- **专业数据库方案**：
 - **键值数据库**：如Redis，通过ID快速查询，性能极高
 - **关系型数据库**：如PostgreSQL，适合存储复杂结构化元数据
 - **文档数据库**：如MongoDB，天然支持JSON格式元数据存储
- **架构解耦优势**
 - **职责分离**：FAISS专注向量检索，元数据交给专业数据库管理
 - **实现示例**：
- **扩展性**：支持不同业务场景下的元数据复杂度需求
- **实现细节说明**
 - **ID验证**：检查retrieved_ids是否为-1（无效ID）
 - **结果展示**：格式化输出排名、距离、原始文本和完整元数据
 - **错误处理**：捕获并提示查询过程中的异常情况
- 向量数据库的选择 02:37:44

向量数据库选择



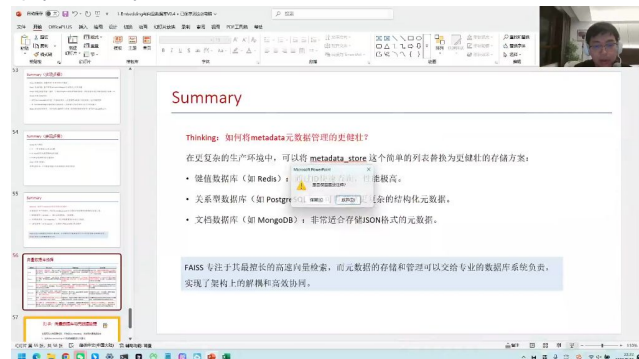
数据库	核心特点	性能表现	适用场景
FAISS	核心算法库，非数据库。由Meta AI开发，提供最广泛、最前沿的ANN索引算法。支持CPU和GPU。	纯粹的性能标杆。在内存中进行向量检索时速度极快，尤其是GPU版本，是衡量其他数据库性能的基准。	算法研究者；需要将向量检索能力深度集成到现有系统中的团队；可以自行构建服务层（API、元数据管理）
Milvus	开源领导者，功能全面。云原生架构，离度可扩展。支持多种ANN索引和丰富的调优参数。	在多个公开的ANN基准测试中，尤其是在大规模数据集上表现优异，吞吐量和延迟控制得非常好。	需要处理海量数据、对性能和扩展性有要求的企业级应用。适合有一定运维能力的团队进行私有化部署。
Pinecone	全托管的商业先驱。主打“Serverless”和易用性，API设计简洁。专注于提供极致的低延迟检索。	性能非常出色，尤其在低延迟方面有优势。由于是托管服务，其内部优化细节不透明，但用户体验感强。	追求快速上线。希望将运维负担降至最低的团队。初创公司和需要快速验证AI应用可行性的场景。
Weaviate	开源，内置数据向量化模块。能够连接各种Embedding模型，实现数据的自动向量化，简化了开发流程。	性能良好，易用性强。其自动向量化功能在开发效率上是一大亮点。	希望简化ETL流程，快速构建从数据到向量再到检索的全链路应用的开发者。
Qdrant	开源，以性能和内存安全著称（使用Rust开发）。过虑查询能力强大且高效。	性能强劲，尤其在过滤条件复杂的混合查询场景下表现突出。内存使用效率高。	对检索性能和资源利用率有极致要求的场景。金融、电商等需要复杂过滤规则的应用。
Elasticsearch	通用搜索巨头，扩展向量检索能力。将向量检索（KNN）作为功能之一。可以无缝结合其强大的全文检索、聚合分析能力。	作为一个通用数据库，其向量检索性能通常弱于专门的向量数据库。但在混合搜索（关键词+向量）场景下，表现优异。	业务需求是以文本搜索为主，向量搜索为辅。希望在一个统一的平台中解决所有搜索问题，而非追求极致的向量检索性能。

① ② ③ ④ ⑤ ⑥ ⑦ ⑧ ⑨ ⑩

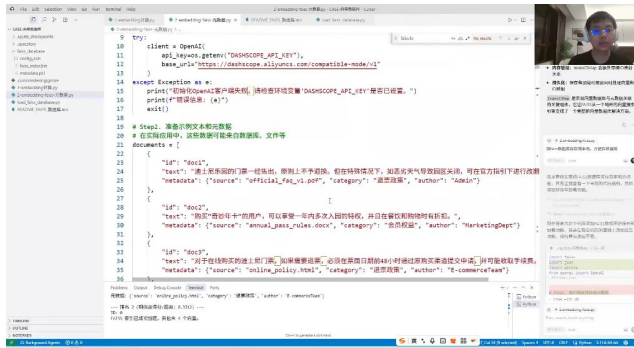
FAISS

- **核心定位**：Meta AI开发的核心算法库，非完整数据库，是衡量其他数据库性能基准
- **性能特点**：CPU/GPU版本检索速度极快，纯粹的内存检索性能标杆
- **适用场景**：
 - 算法研究者需要前沿ANN索引算法
 - 需要将向量检索能力深度集成到现有系统的团队
 - 需自行构建服务层（API、元数据管理）的场景

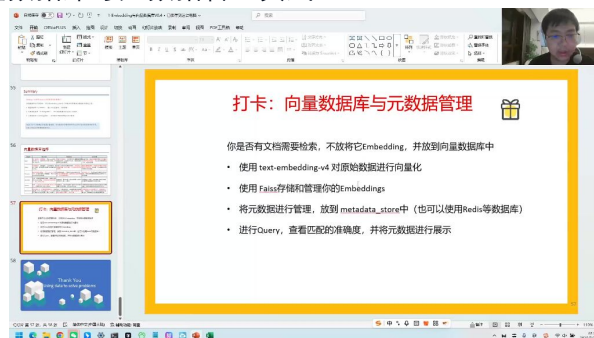
- 持久化方法：
 - 通过pickle.dump()保存index对象
 - 加载时使用pickle.load()和faiss.read_index()
 - 生成三个文件：config（配置）、bin（索引数据）、metadata（pickle序列化数据）
- Milvus
 - 核心特点：Go语言开发的开源领导者，分布式吞吐能力强
 - 性能表现：在大规模数据集ANN基准测试中表现优异
 - 适用场景：处理海量数据且对性能/扩展性要求高的企业级应用
- Pinecone
 - 架构特点：全托管闭源商业服务，主打Serverless和易用性
 - 性能优势：低延迟检索表现突出（内部优化细节不透明）
 - 适用场景：需要快速上线、最小化运维负担的团队（如初创公司验证AI应用）
- Weaviate
 - 核心功能：内置数据向量化模块，可连接多种Embedding模型
 - 开发优势：自动向量化功能显著提升开发效率
 - 适用场景：需要快速构建从原始数据到向量检索全链路的开发者
- Qdrant
 - 技术特点：Rust开发，以性能和内存安全著称
 - 查询优势：过滤条件复杂的混合查询场景表现突出
 - 适用场景：金融/电商等需要复杂过滤规则的高性能应用
- Elasticsearch
 - 功能定位：通用搜索引擎扩展向量检索能力（KNN作为功能之一）
 - 混合搜索：在"关键词+向量"混合搜索场景表现优异
 - 适用场景：以文本搜索为主、向量搜索为辅的统一搜索平台需求
- 元数据管理方案



- 架构原则：FAISS专注向量检索，元数据交给专业数据库管理
- 存储方案：
 - 键值数据库（如Redis）：高性能简单元数据
 - 关系型数据库（如PostgreSQL）：复杂结构化元数据
 - 文档数据库（如MongoDB）：JSON格式元数据
- 更新机制：
 - 原始文本变更时才需更新向量
 - 仅元数据属性变更（如文件名、作者）不影响向量
- 关键技术实现



- IndexIDMap作用：实现向量与元数据ID映射的关键组件
- 多模态支持：
 - 可建立多个FAISS库（如文本库和图像库）
 - 图像处理推荐使用CLIP等跨模态Embedding模型
- 业务逻辑：
 - 向量用于相似度计算筛选片段
 - 元数据保存原始文本供大模型推理
 - 溯源时需要元数据标明信息来源
- 数据预处理：
 - 多种格式文件（PDF/Word/PPT）需先转为文本
 - 通过切分等处理后再存入向量数据库
- 案例分析与实践 02:52:18
 - 向量数据库与元数据管理实践



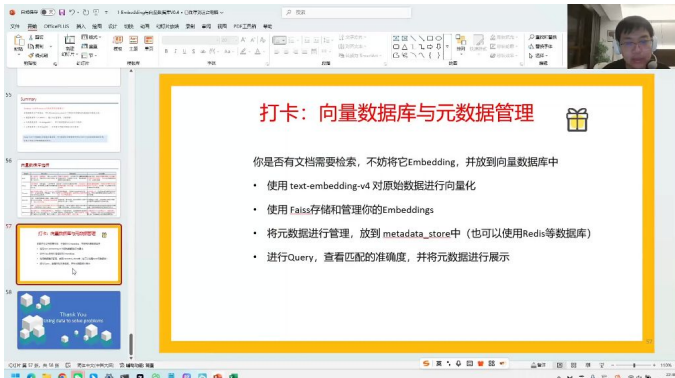
- 文本向量化: 使用text-embedding-v4模型将原始文档转化为向量表示
- 向量存储: 推荐使用Faiss进行向量存储和管理，因其轻量级且检索速度快
- 元数据管理: 可将元数据存储于metadata_store或Redis等数据库中
- 查询流程: 通过向量相似度匹配检索内容，并关联展示对应的元数据信息
- 三国演义文本Embedding实践
 - 数据处理: 使用word2vec工具对《三国演义》文本进行向量化处理
 - 模型选择: 基础实践可用简单模型，进阶学习可选用更大的预训练模型
 - 文件来源: 课程提供three_kingdoms源文件作为实践数据
- 向量数据库技术选型



- **FAISS:**
 - 特点: Meta开发的算法库, 支持CPU/GPU, 提供前沿ANN算法
 - 适用场景: 需要深度集成向量检索能力的算法研究场景
- **Milvus:**
 - 特点: 开源云原生架构, 支持多种ANN索引和参数调优
 - 适用场景: 大规模数据处理的企业级应用
- **Pinecone:**
 - 特点: 全托管服务, API简洁, 低延迟检索
 - 适用场景: 快速验证AI应用的初创公司
- **Weaviate:**
 - 特点: 内置自动向量化模块, 简化开发流程
 - 适用场景: 需要快速构建全链路应用的场景
- **Qdrant:**
 - 特点: Rust开发, 内存安全, 复杂过滤查询能力强
 - 适用场景: 对性能有极致要求的金融/电商场景
- Embedding技术问答
 - **与LLM的区别:**
 - 功能差异: Embedding专注语义向量化, LLM侧重生成和理解
 - 成本差异: Embedding模型通常计算成本更低
 - **专业模型训练:**
 - 流程: 使用word2vec或BERT在专业语料上训练
 - 部署: 可打包为可下载的模式文件
 - **维度说明:**
 - 处理过程: 原始句子→分词→压缩到向量空间 (如1024维)
 - **推荐系统应用:**
 - LLM作用: 用于高级语义理解和生成, 与DeepFM等专有模型配合使用
 - **元数据必要性:**
 - 向量作用: 相似度计算和片段筛选
 - 元数据作用: 保存原始文本供LLM进一步处理
- 图像向量化工具
 - **工具推荐:**
 - CLIP: 通用的图像-文本跨模态理解工具
 - DINOv2: Facebook开发的图像特征提取工具
 - **使用方式:** 传入图片获取向量表示, 后续处理流程与文本类似
 - **人脸特征:**
 - 专用工具: 需使用专门的人脸特征提取库
 - 模型选择: 不同工具使用不同训练语料, 如dash scope的text-embedding-v4专攻中文

8. 课程总结 02:56:19

1) 向量数据库与Embedding技术



- - **核心概念：** 向量数据库是管理大量向量数据的专用数据库，作为大模型的"小助手"负责数据筛选工作
 - **数据处理流程：**
 - 使用 *text - embedding - v4* 对原始文本进行向量化
 - 将生成的 Embedding 向量存入 Faiss 等向量数据库
 - 通过元数据管理(如 Redis)存储关联信息
 - **应用价值：** 有效过滤无意义词汇（如英文中的"a/the/on/off"），提升大模型输入质量
- 2) 实践要点
- **训练选择：** 需要根据具体场景选择合适的 Embedding 模型进行训练
 - **元数据管理：** 建议将元数据存储于专门的 metadata_store 中，便于后续查询验证
 - **效果验证：** 通过 Query 测试匹配准确度，并可视化展示元数据关联结果
- 3) 学习建议
- **练习重点：** 聚焦酒店案例中的两个典型场景进行实践
 - **学习路径：**
 - 初期可先理解整体逻辑框架
 - 数学原理理解难度较大时可暂缓深入
 - 通过实际任务驱动逐步掌握核心技术

三、知识小结

知识点	核心内容	关键点	应用场景
Embedding概念	将高维数据压缩到低维空间的技术	- 解决稀疏性问题 - 26000维→1024维降维 - 语义特征保留	推荐系统、语义搜索、知识库构建
N元语法	文本特征提取方法	- 一元(单词)、二元(词组)、三元(组合) - TF-IDF计算特征权重 - 解决词序问题	酒店描述特征分析、商品标签生成
余弦相似度	向量相似度计算方法	- 夹角越小相似度越高 (1→0→-1) - 公式： $\cos\theta=(A\cdot B)/(A B)$ - 语义相关性量化	句子相似度比较、推荐排序
Word2Vec	词向量训练工具	- 神经网络预测邻居词 - 三国演义案例训练 - 加减法语义运算(曹操+刘备-张飞)	文本特征提取、语义关系发现

向量数据库	高效相似检索系统	- Faiss(最快)、ES(混合搜索)、Milvus(分布式) - 十亿级数据毫秒响应 - 元数据管理(来源/作者)	知识库检索、跨模态搜索、实时推荐
RAG架构	检索增强生成技术	- 向量库筛选TOP5片段 - 作为上下文喂给LLM - 解决窗口限制问题	企业知识问答、精准报告生成
模型选型	Embedding模型选择	- 中文优选BGE - 多语言选paraphrase-multilingual - 套娃模型(齐娜AI)	跨境电商、多语言客服系统
性能优化	工程实践要点	- 768维性价比平衡点 - 测试集构建金标准 - CPU版Faiss轻量部署	社交媒体实时推荐、金融风控系统